

Dossier d'Analyse - Épreuve E6

TW4 - Projet Ballon Stratosphérique



Lycée
TOUCHARD
WASHINGTON

Projet réalisé par : **PAIN Iako**
BRANDS Noé
KALO Harold
GUERRIAU Lucien

Formation initiale du **BTS CIEL-IR**
2ème année à Le Mans

Projet supervisé par : **LE CREN Anthony**
KHAMLACH Jilali

Date de rendu prévue le 8 Juin 2026

Table des Matières

1.1. Introduction.....	3
1.2. Objectifs.....	3
1.3. Répartition des Tâches.....	4
Synoptique de la nacelle.....	14
2.3 Description des capteurs et des actionneurs.....	15
Synoptique de la station au sol.....	17
Synoptique de la nacelle.....	18
2.4 Dictionnaire des Variables.....	19
2.5 Récolte des informations d'APRS.fi & OSM.....	21
2.6. Trames JSON.....	21
3.1. Cas d'Utilisations.....	22
3.2. Dictionnaire Acteurs.....	23
Etudiant 1.....	24
Cas d'Utilisation N°1 : Visualiser la télémétrie [NACELLE].....	24
Cas d'Utilisation N°2 : Recevoir la télémétrie [SOL].....	25
Cas d'Utilisation N°3 : Émettre la télémétrie [NACELLE].....	26
Cas d'Utilisation N°4 : Obtenir les grandeurs physiques [NACELLE].....	27
Etudiant 2.....	28
Cas d'Utilisation N°1 : Envoyer une requête [Sol].....	28
Cas d'Utilisation N°2 : Répondre à la requête.....	30
Cas d'Utilisation N°3 : Lire et afficher la réponse.....	32
Cas d'Utilisation N°4 : Détecter éclatement & atterrissage.....	34
Cas d'Utilisation N°5 : Envoyer message éclatement & atterrissage.....	36
Etudiant 3.....	37
Cas d'Utilisation N°1 : Prendre une photo du ballon [NACELLE].....	37
Cas d'Utilisation N°2 : Enregistrer la photo sur la carte SD [NACELLE].....	38
Cas d'Utilisation N°3 : Moduler le signal HF [NACELLE].....	39
Cas d'Utilisation N°4 : Transmettre la photo au sol [SOL].....	40
Cas d'Utilisation N°5 : Importer les photos enregistrées [SOL].....	41
Cas d'Utilisation N°6 : Décoder le signal HF [SOL].....	42
Cas d'Utilisation N°7 : Enregistrer une photo [SOL].....	43
Cas d'Utilisation N°8 : Visualiser une photo [SOL].....	44
Cas d'Utilisation N°9 : Générer une vidéo à partir des photos [SOL].....	45
Etudiant 4.....	46
Cas d'Utilisation N°1 : Visualiser la position du véhicule par rapport au ballon.....	46
Cas d'Utilisation N°2 et 3: Mettre à jour la base de donnée & Récupérer les positions.....	47
3.3. Matrice RACI.....	48
Définition des Rôles RACI.....	48
3.4. Diagrammes de Classes.....	49
4.1. Interface Internaute.....	60
4.2. Interface Opérateur.....	61
5.1. Diagramme MCD (Modèle Conceptuel de Données).....	62
6.1. Diagramme de Gantt.....	63

1. Analyse Fonctionnelle

1.1. Introduction

Le présent dossier expose un projet visant au développement d'un ballon stratosphérique **autonome**. Celui-ci sera doté d'une nacelle embarquée, capable d'assurer la capture d'images, la collecte de données de télémétrie et la communication avec une station au sol. Ce projet s'inscrit dans le cadre d'un BTS CIEL réalisé au Lycée Touchard-Washington et intègre des disciplines telles que l'électronique, les télécommunications (SSTV, LoRa, APRS), l'informatique embarquée et les systèmes web. L'objectif est de permettre un suivi scientifique et la localisation en temps réel du ballon à très haute altitude. Bénéficiant d'un financement du CNES et de l'association Planète Sciences, ce travail poursuit des objectifs de recherche (vérification de la loi des gaz parfaits) et de démonstration technologique (établissement d'une liaison LoRa bidirectionnelle).

L'architecture proposée repose sur l'intégration d'un raspberry pi à bord de la nacelle, complétée par un ensemble complet au sol. Le système comprend une caméra pour la prise d'images (transmises via SSTV), des capteurs dédiés à la télémétrie (mesure de la température, de la pression et de l'accélération), un module LoRa pour les échanges de données, ainsi qu'un système web de suivi incluant une base de données, un décodeur SSTV et une interface cartographique. Ce dossier d'analyse présente de manière exhaustive la démarche méthodologique adoptée : analyse fonctionnelle, définition des cas d'utilisation, spécifications techniques détaillées et répartition des responsabilités entre les quatre étudiants impliqués. Cette préparation rigoureuse est essentielle pour garantir le succès du lancement, dont la date est fixée au 18 mai 2026.

1.2. Objectifs

Le présent projet vise plusieurs objectifs fondamentaux : premièrement, assurer le **suivi en temps réel** du ballon stratosphérique durant son ascension et son vol par l'exploitation d'une liaison radio (LoRa/APRS) couplée à un système de cartographie en ligne ; deuxièmement, collecter des **données scientifiques** exhaustives (température, pression, accélération) en haute altitude, permettant l'étude de l'atmosphère et la vérification expérimentale de la loi des gaz parfaits ; troisièmement, garantir le succès de la **récupération** de la nacelle et de son contenu (incluant photographies et télémesures de haute résolution) grâce à un dispositif de localisation précis ; enfin, démontrer la **faisabilité technique** de l'intégration d'un système embarqué autonome apte à gérer l'acquisition de données, la transmission de télémesures et d'images (SSTV) dans des conditions environnementales extrêmes.



1.3. Répartition des Tâches

Le tableau ci-dessous servira à documenter la répartition initiale des tâches entre les membres de l'équipe :

Étudiant	Fonctions à développer et tâches à effectuer	
Étudiant 1 (Iako)	Liste des fonctions assurées par l'étudiant : Développement du module de télémesure Sauvegarde locale dans un fichier CSV Rédaction de la documentation utilisateur online	Installation : Chaîne de développement sur netbeans
		Mise en œuvre : Développement C++ sur Raspberry
		Configuration : raspbian SSH I2C
		Réalisation : Les cas d'utilisation en charge
		Documentation : Responsable du Guide utilisateur
		Logiciel : ide netbeans
		Matériel : carte ballon raspberry pi, carte SD, PA Wifi



Étudiant	Fonctions à développer et tâches à effectuer	
Étudiant 2 (Noé)	Liste des fonctions assurées par l'étudiant : Développement de l'interface utilisateur permettant l'envoi de requêtes Détection la chute libre Détection l'atterrissage Envoyer des messages Communication Lora montante et descendante.	Installation : Chaîne de développement sur netbeans
		Mise en œuvre : Développement C++ sur PC
		Réalisation : Les cas d'utilisation en charge
		Documentation : Responsable du Guide utilisateur
		Logiciel : netbeans
		Matériel : carte ballon raspberry pi, carte SD, PA Wifi, gateway lora



Étudiant	Fonctions à développer et tâches à effectuer	
Étudiant 3 (Harold)	Liste des fonctions assurées par l'étudiant : transmettre les photos Sauvegarde des images sur la carte SD selon un format compressé Décoder les photos reçus en SSTV Interface Web affichage des photos reçues Création d'une vidéo à partir des photos du vol Rédaction de la documentation utilisateur online	Installation : Chaîne de développement sur netbeans
		Mise en œuvre : Développement C++ sous linux raspbian
		Configuration : raspbian SSH I2C SPI Camera
		Réalisation : Les cas d'utilisation en charge
		Documentation : Responsable du Guide utilisateur
		Logiciel : ide netbeans
		Matériel : Serveur linux apache

Étudiant	Fonctions à développer et tâches à effectuer	
Étudiant 4 (Lucien)	Liste des fonctions assurées par l'étudiant : Développement de l'application permettant de récupérer la position du véhicule via APRS.fi Développement de l'application permettant la visualisation du véhicule par rapport au ballon Installation de la base de données Élaborer et configurer la BD et le serveur de BD Rédaction de la documentation utilisateur online.	Installation : Chaîne de développement sur netbeans
		Matériel : Serveur linux apache

2. Spécifications Techniques

Entrées

Grandeur physique	Échelle de mesure	Unité	Précision	Type de grandeur	Capteur	Description
Horodatage	0 à 3155760000	Sec- onde	Sans Unité	Numérique (I2C)	RTC DS3231	Horodatage les données
Température	-40 à 85	°C	Dixième	Numérique (I2C)	BME280	Mesure la température
Température interne	-55 à 125	°C	Dixième	Numérique (I2C)	LM75	Mesure la température interne
Pression atmosphérique	300 à 1100	hPa	Unité	Numérique (I2C)	BME280	Mesure la pression atmosphérique
Humidité	0 à 100	%	Dixième	Numérique (I2C)	BME280	Mesure l'humidité

Entrées

Grandeur physique	Échelle de mesure	Unité	Précision	Type de grandeur	Capteur	Description
Intensité signal (RSSI)	-120 à -50	dBm	Unité	Numérique (SPI)	Récepteur LoRA	Mesure la puissance du signal LoRa reçu.
Accélération	-160 à 160	m/s ²	Dixième	Numérique (Calculé)	HW 123	Accélération X Y et Z du ballon
Position	0 à 360	°	millionième	Numérique (Calculé)	SIRFstar IV	Mesure la position du ballon
Rapport Signal/Bruit (SNR)	-20 à 10	dB	Unité	Numérique (SPI)	Récepteur LoRA	Mesure de la qualité de la liaison LoRa reçue.

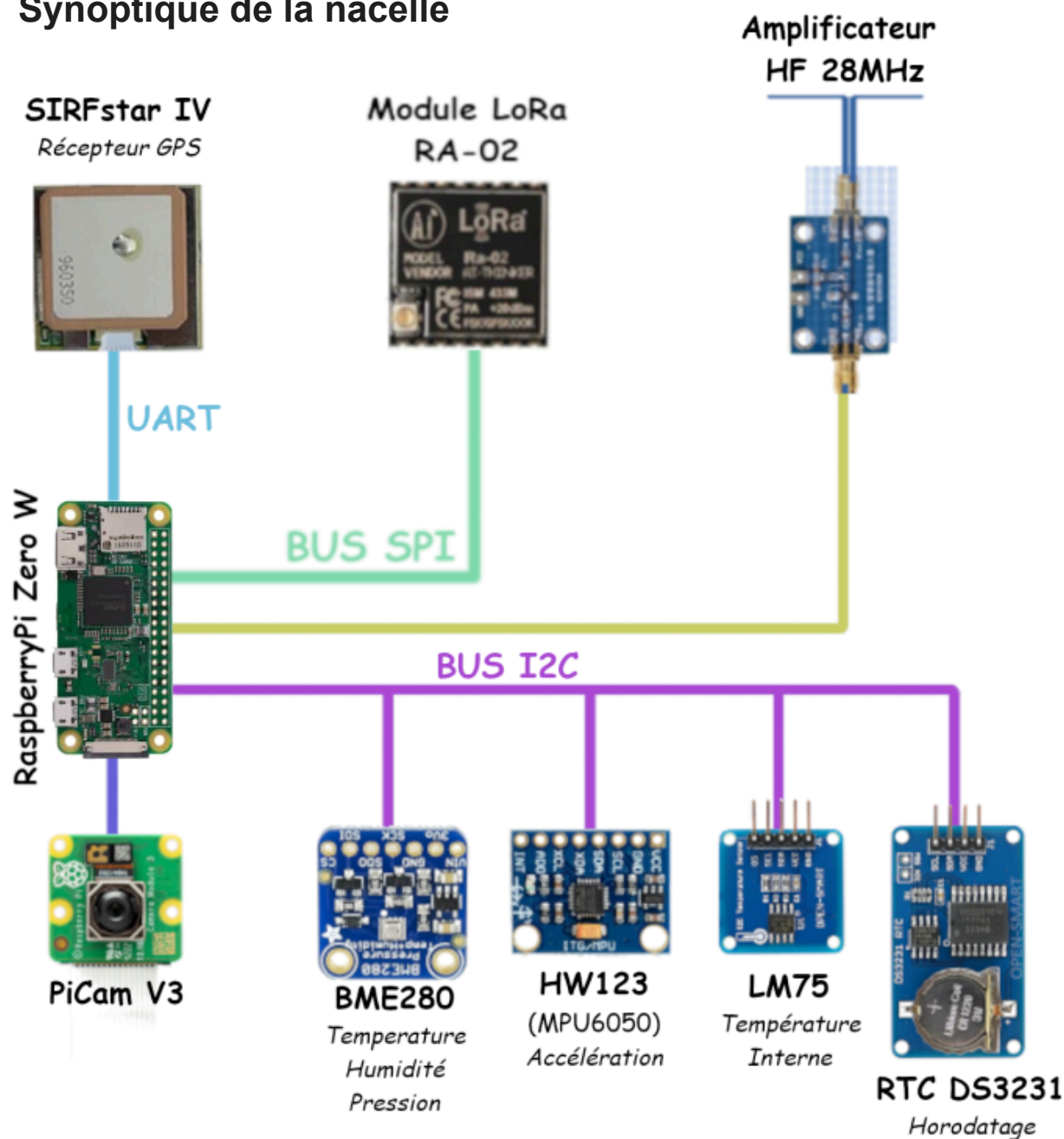
Entrées						
Grandeur physique	Échelle de mesure	Unité	Précision	Type de grandeur	Capteur	Description
Trame APRS (LoRa)	20 à 40	octet	Sans Unité	Numérique (SPI)	Récepteur LoRa	Trame de données APRS LoRa contenant la télémétrie du ballon.
Sorties						
Émetteur LoRa (Transmission)	Trame de données	N/A	Numérique (HF)	Bus SPI (Registres & FIFO)	SPI	Émission d'une trame APRS à 433 MHz.

Entrées						
Grandeur physique	Échelle de mesure	Unité	Précision	Type de grandeur	Capteur	Description
Lumière (image)	8	Mega Pixels	Unité	Numérique (CSI)	TNBA 1392	Capture des données brutes du capteur 8 MP via le bus haute vitesse CSI pour traitement logiciel.
Onde Radio	29	MHz	Dixième	Analogique	Emetteur VHS FM	Porteuse haute fréquence (VHF) permettant de transporter le signal SSTV sur de longues distances.
Signal Audio	1200 à 2300	Hz	Unité	Analogique	Carte Son (PC)	Modulation de fréquence représentant les lignes de pixels et la synchronisation de l'image.
Sorties						
Signal Modulé (SSTV)	1200 à 2300	Hz	Analogique	Protocole Martin M1	Jack/PWM	Conversion sonore des données pixels en fréquences audibles
Émetteur HF	29	MHz	Analogique (HF)	Modulation FM	Antenne	Rayonnement de l'onde radio porteuse pour transmettre l'image vers le sol.

Sorties						
Grandeur physique	Échelle de mesure	Unité	Précision	Type de grandeur	Capteur	Description
Carte SD (Stockage)	64	Go	Numérique	Bus Interne (Système de fichiers)	SPI	Stockage des données télémétriques. Écriture fichier CSV

Sorties						
Grandeur physique	Échelle de mesure	Unité	Précision	Type de grandeur	Capteur	Description
Écran OLED (Affichage)	128x64	Pixel	Numérique	Bus I2C (Adresses & Données)	I2C	Affichage des données textuelles lors des tests de réception des données.
LED de statut (Indication)	État Binaire (0 ou 1)	-	Numérique (tout ou rien)	Tension 0 - 3.3 V (GPIO)	GPIO	Indication visuelle de l'état opérationnel du système lors des tests.
Interface Web	-	N/A	Numérique	TCP/IP(HTTP)	Ethernet/WIFI	Envoi des données de l'image pour affichage sur le poste de contrôle

Synoptique de la nacelle



2.3 Description des capteurs et de la transmission (Nacelle)

Capteur environnement – BME280

Rôle	Mesurer la température, la pression atmosphérique et l'humidité relative dans/près de la nacelle pendant le vol.
Caractéristiques	Mesure de température, pression et humidité dans un seul composant. Interface numérique I2C ou SPI Alimentation typique 3,3 V. Plage de mesure typique : Température : environ -40 °C à +85 °C. Pression : environ 300 à 1100 hPa. Humidité : 0 à 100%.

Capteur d'accélération – HW 123

Rôle	Mesurer les accélérations sur les axes X, Y et Z pour détecter les phases de vol (montée, chute libre, atterrissage) et enregistrer les efforts subis par la nacelle.
Caractéristiques	Accéléromètre 3 axes. Interface numérique Alimentation de 3,3 V Plage de mesure typique : ± 16 g, soit environ ± 160 m/s ² .

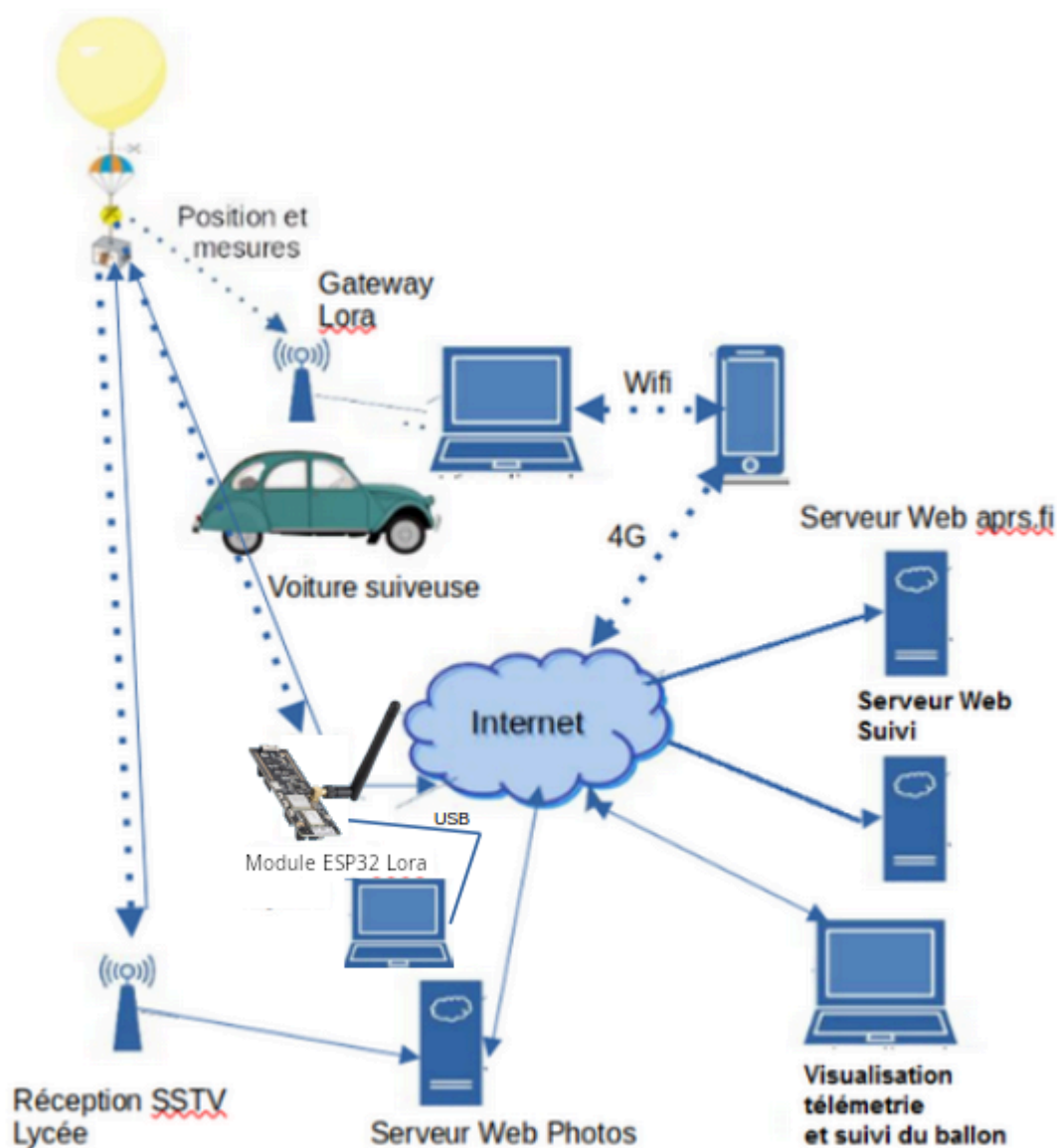
Module d'horodatage – RTC DS3231

Rôle	Garantir un horodatage (timestamping) précis et constant de toutes les données collectées. Il permet de corréliser chaque mesure (pression, altitude, accélération) à une heure exacte, même si la raspberry pi subit une coupure d'alimentation ou un redémarrage en vol.
Caractéristiques	Horloge Temps Réel (RTC) de haute précision. Interface : Numérique (I2C). Compensation thermique : Intègre un oscillateur à quartz compensé en température (TCXO) pour rester précis malgré le froid extrême de la haute atmosphère. Alimentation : 3,3 V (compatible Raspberry Pi) avec emplacement pour pile de secours (CR2032) afin de conserver l'heure hors tension.

Capteur de température – LM75

Rôle	Mesurer la température ambiante à l'intérieur ou à l'extérieur de la nacelle. Ces données servent à surveiller la santé thermique des composants électroniques et à enregistrer le profil de température de l'atmosphère durant l'ascension.
Caractéristiques	Capteur de température numérique. Interface : Numérique (I2C). Jusqu'à 8 capteurs peuvent être connectés sur le même bus grâce à des adresses configurables. Plage de mesure : -55 °C à +125 °C. Précision : ± 2 °C sur la plage de -25 °C à +100 °C. Résolution : 0,125 °C (conversion sur 9 bits).

Récepteur GPS – SIRFstar IV	
Rôle	Fournir la position du ballon (latitude, longitude, altitude) et l'heure pour horodater correctement les mesures.
Caractéristiques	Récepteur GNSS utilisant le chipset SIRFstar IV. Sortie de trames NMEA sur interface série (UART). Précision de position de l'ordre de quelques mètres (en conditions normales). Alimentation typique 3,3 V / 5 V suivant le module.
Module radio LoRa – RA-02	
Rôle	Transmettre les trames de télémétrie (mesures + événements) du ballon vers la station sol via la liaison LoRa.
Caractéristiques	Modulation LoRa en bande 433 MHz. Interface SPI avec la raspberry pi Puissance d'émission et paramètres radio configurables par logiciel. Utilisé pour envoyer périodiquement la télémétrie au format APRS Weather.
Carte SD	
Rôle	Enregistrer localement les mesures (fichiers CSV) et les images capturées, afin de conserver les données même en cas de perte de liaison radio.
Caractéristiques	Interface SPI avec la raspberry pi Support des systèmes de fichiers de type FAT32. Permet l'enregistrement horodaté des mesures toutes les 20s pendant le vol.
Capteur d'image - Caméra PI module 3 (TNBA1392)	
Rôle	Capter le flux lumineux pour créer une image numérique lors de l'appel.
Caractéristiques	8 Mégapixels. Interface CSI.
Conditionneur de signal - Émetteur HF BLU	
Rôle	Adapter le signal audio SSTV en ondes électromagnétiques pour la transmission.
Caractéristiques	Sortie Audio. Ondes HF. Fréquence modulée pour transport du signal SSTV



Synoptique de la station au sol

2.4 Dictionnaire des Variables

Nom	Type	Précision	Taille	Description	Exemple
timestamp	Entier int	[1] Unité	10	Horodatage	1773035785
temperature	Réel float	[0.01] Centième	6	Température, positive ou négative	-32.5
pression	Réel float	[0.1] Dixième	6	Pression (en hPa)	1013.2
humidité	Entier int	[1] Unité	3	Humidité (en %)	15
accelerationX	Réel float	[0.01] Centième	6	accélération (en m/s)	0.15
accelerationY	Réel float	[0.01] Centième	6		-0.02
accelerationZ	Réel float	[0.01] Centième	6		9.81
longitude	Réel float	[0.00001] Cent-Millionième	10	Longitude (en degrés)	2.35220
latitude	Réel float	0.00001] Cent-Millionième	10	Latitude (en degrés)	48.85660
altitude	Réel float	[1] Unité	7	Altitude (en m)	1250.0
trameAPRS	Réel float	Sans Unité	255	Trame de données au format APRS (Automatic Packet Reporting System).	\$GPGLGA...
rsi_dernier	Entier int	[1] Unité	3	Dernière valeur RSSI reçue (en dBm).	-85
snr_dernier	Entier int	[1] Unité	3	Dernière valeur SNR reçue (en dB).	8
flagBURST	Booléen bool	Sans Unité	1	Indicateur de l'événement "Burst" (éclatement du ballon). V si le burst a eu lieu.	F
flagLANDING	Booléen bool	Sans Unité	1	Indicateur de l'événement "Landing" (atterrissage). V si l'atterrissage a eu lieu.	V
timestamp_burst	Entier int	[1] Unité	10	Horodatage de l'événement "Burst".	3600500
timestamp_landing	Entier int	[1] Unité	10	Horodatage de l'événement "Landing".	7200000
altitude_burst	Réel float	[0.1] Dixième	8	Altitude enregistrée lors de l'événement "Burst" (en mètres, avec une précision au dixième).	32000.0
altitude_landing	Réel float	[0.1] Dixième	8	Altitude enregistrée lors de l'événement "Landing" (en mètres, avec une précision au dixième).	150.0

callsign	Chaîne char	Sans Unité	10	Indicatif radio (indicatif d'appel) du ballon.	F6HTB
taux_echantillonnage	Entier int	[1] Unité	3	Taux auquel les données de télémétrie sont échantillonnées et transmises.	50
RSSI	Entier int	[1] Unité	3	Valeur RSSI (Received Signal Strength Indicator) actuelle (en dBm).	-92
SNR	Entier int	[1] Unité	2	Valeur SNR (Signal-to-Noise Ratio) actuelle (en dB).	6
resultatEmission	Entier int	Sans Unité	1	Code de retour indiquant le succès (1) ou l'échec (0) de la dernière tentative d'émission LoRa (ex: 1 pour succès, 0 pour échec).	1
flux_JPEG	Binaire	[1] Unité	+Mo	Signal envoyé par la Caméra vers la raspberry pi après l'acquisition.	[Données RAW]
liste_fichiers (JPEG)	Binaire	Sans Unité	~ 150 Ko	Fichier image (JPEG) transféré pour la modulation ou le web.	"photo.jpg"
signal_pret	Booléen bool	Sans Unité	1	Message de retour indiquant que la modulation SSTV est finie	V
id_photo	Entier int	[1] Unité	4	Identifiant unique de la capture dans la base	1024
succès_importation	Booléen bool	Sans Unité	1	Confirmation finale de la mise à jour de la base de données.	V
confirmation_ecriture	Booléen bool	Sans Unité	1	Acquittement renvoyé par la carte SD après le stockage.	V
chemin_acces	Chaîne char	Sans Unité	30	Retour de l'emplacement du fichier enregistré sur la SD.	"/images/vol 01.jpg"
publication_OK	Booléen bool	Sans Unité	1	Acquittement du serveur confirmant l'affichage (F2-IHM).	V
donnees_pixels	Binaire	Sans Unité	+Mo	Flux de données brutes envoyé par la caméra (Acquisition).	[Pixels RAW]
ecriture_terminee	Booléen bool	Sans Unité	1	Retour de la carte SD confirmant que le fichier est bien enregistré.	V
confirmation_ID	Entier int	[1] Unité	4	Numéro unique attribué à la photo par le système de gestion.	1024
onde_sinusoidal	Entier int	[1] Unité	4	Fréquence audio calculée pour la trame SSTV (1200 à 2300 Hz).	1500
affichage_en_cours	Booléen bool	Sans Unité	1	État renvoyé par le serveur web pendant le chargement de l'image.	V
afficher_image	Chaîne char	Sans Unité	30	Lien (URL/Chemin) envoyé à l'interface pour afficher la photo finale.	"/img/photo1.jpg"

2.5 Récolte des informations d'[APRS.fi](#) & OSM

[APRS.fi](#): Carte interactive qui permet de suivre en temps réel la position, la météo et les messages partagés par les radioamateurs du monde entier.

OMS: Carte du monde collaborative et gratuite, alimentée par des bénévoles et souvent décrite comme le « Wikipédia de la cartographie ».

Le but ici est de récupérer la position du ballon et du véhicule suiveur qui sont affiliés à un indicatif radioamateur afin d'ensuite placer la position du ballon sur une carte OSM

2.6. Trames JSON

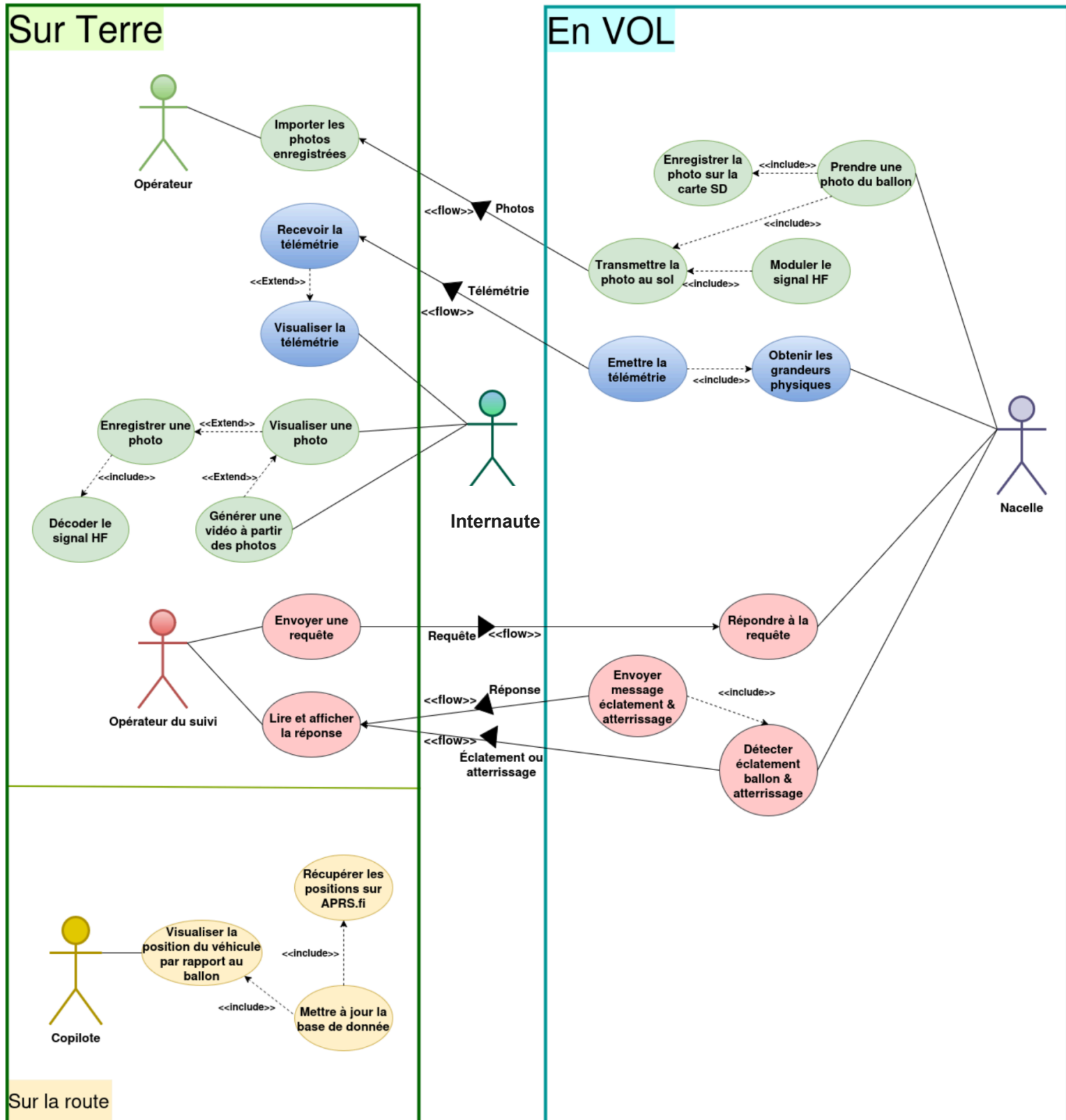
Les structures JSON employées pour la communication inter-composants (par exemple, de l'objet connecté vers le serveur ou de l'API vers l'interface utilisateur) seront spécifiées dans cette section.

```
{
  "command": "get",
  "result": "ok",
  "what": "loc",
  "found": 1,
  "entries": [
    {
      "name": "OH7RDA",
      "type": "I",
      "time": "1267445689",
      "lasttime": "1270580127",
      "lat": "63.06717",
      "lng": "27.66050",
      "symbol": "V#",
      "srcall": "OH7RDA",
      "dstcall": "APND12",
      "phg": "44603",
      "comment": "VR,W,Wn,Tn Siilinjarvi",
      "path": "WIDE2-2,qAR,OH7AA"
    }
  ]
}
```

3. Conception UML

3.1. Cas d'Utilisations

Pour chaque cas d'utilisation, fournir une description, les pré-conditions, les post-conditions et le diagramme de séquence associé.

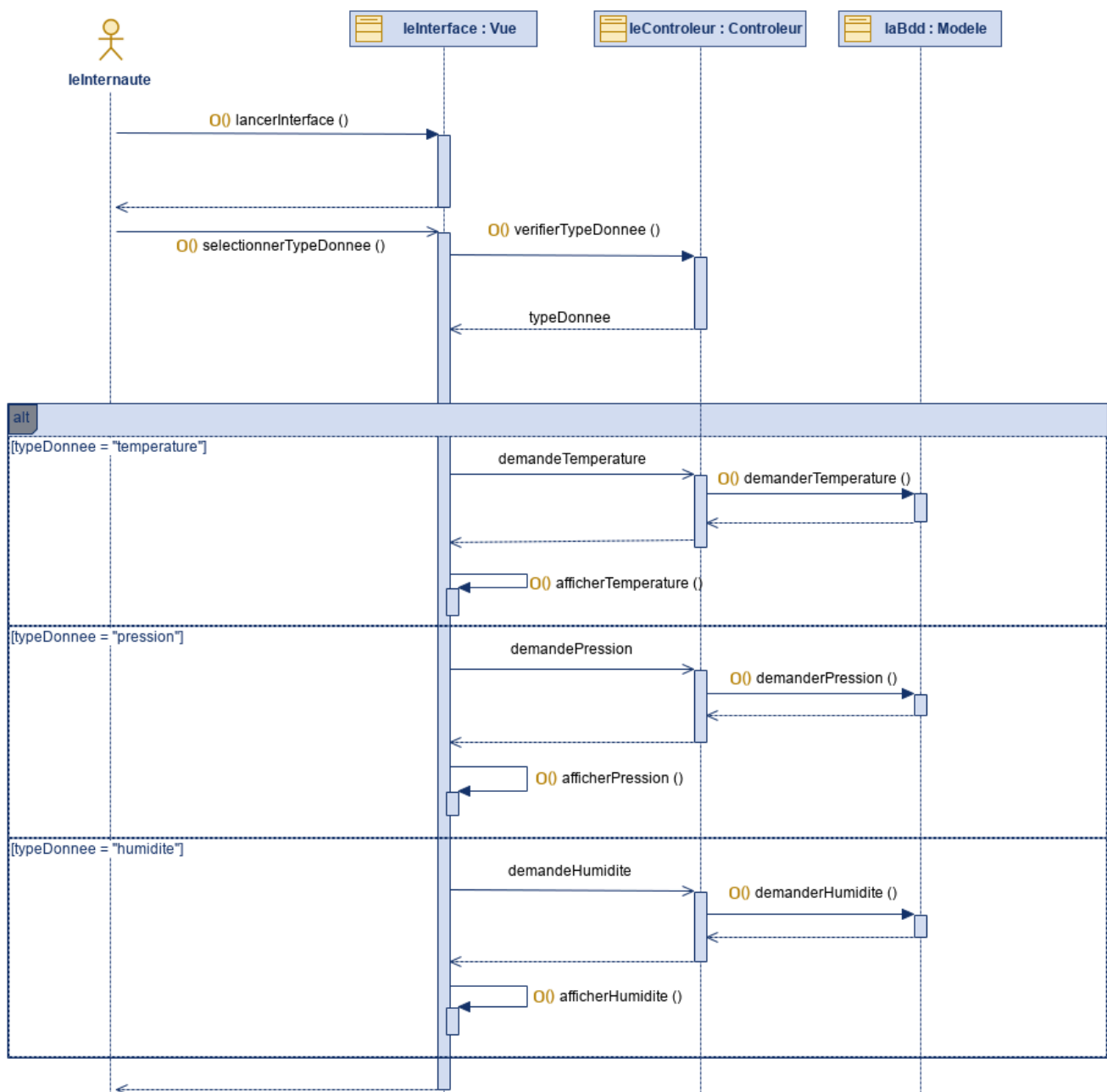


3.2. Dictionnaire Acteurs

Acteurs du Projet et Leurs Rôles		
Nom de l'Acteur	Type	Description / Rôle Principal
Ballon (Nacelle)	Système	Acteur principal (du point de vue de la station sol). Assure l'autonomie en vol, effectue les mesures scientifiques (Pression, Température, Accélération), capture les images, détecte les événements critiques (éclatement, atterrissage) et transmet les données (LoRa et SSTV). Répond également aux commandes depuis le sol.
Opérateur du suivi	Humain	Opère depuis le véhicule suiveur ou le lycée. Utilise la liaison LoRa montante pour solliciter la nacelle (demande RSSI/SNR) et ajuste les paramètres de transmission (ex. : fréquence de prise de photos).
Opérateur d'analyse	Humain	Intervient majoritairement post-vol. Récupère la carte SD de la nacelle pour l'importation des photos haute résolution et du fichier de télémétrie CSV, en vue de l'analyse scientifique des résultats.
Internaute	Humain	Utilisateur distant se connectant au serveur Web du projet. Consulte en temps réel la position du ballon sur une carte, les photos décodées et les données de télémétrie.
Copilote	Humain	Passager du véhicule suiveur. Utilise une application dédiée pour visualiser la position du véhicule par rapport au ballon et guider le chauffeur vers le point de récupération.
Véhicule suiveur	Système / Entité	Voiture équipée d'une Gateway LoRa et d'un point d'accès Wi-Fi. Essentiel pour la réception des trames de données à faible altitude et lors de l'atterrissage.
APRS.fi	Système Externe	Service web mondial pour le suivi APRS. Reçoit les paquets de position via la Gateway LoRa, permettant ainsi l'affichage du ballon sur la cartographie internationale.

Cas d'Utilisation N°1 : Visualiser la télémétrie [NACELLE]

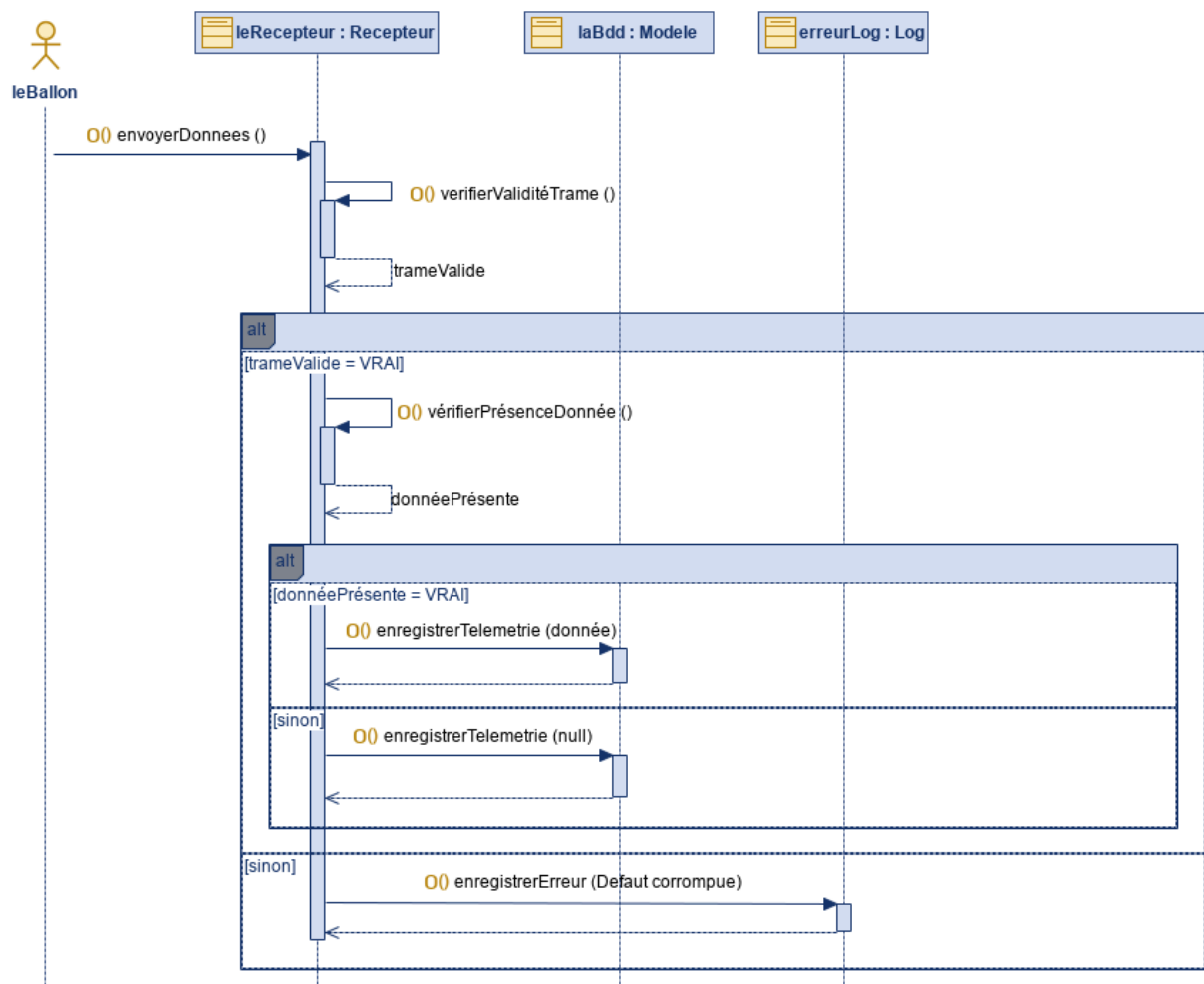
- **Acteur :**
 - Internaute
- **Classes :**
 - Vue
 - Contrôleur
 - Modèle
- **Pré-condition :**
 - L'interface web est lancée sur le navigateur de l'internaute
- **Scénario Principal :**
 - L'internaute lance l'interface, puis sélectionne le type de données à voir, selon le choix, les valeurs stockées dans la bdd pour la température, la pression ou l'humidité est affichée.
- **Scénario d'Erreur :**
 - Si la demande a la bdd ne fonctionne pas, afficher un message d'erreur.



Cas d'Utilisation N°2 : Recevoir la télémétrie [SOL]

Description : Recevoir la télémétrie.

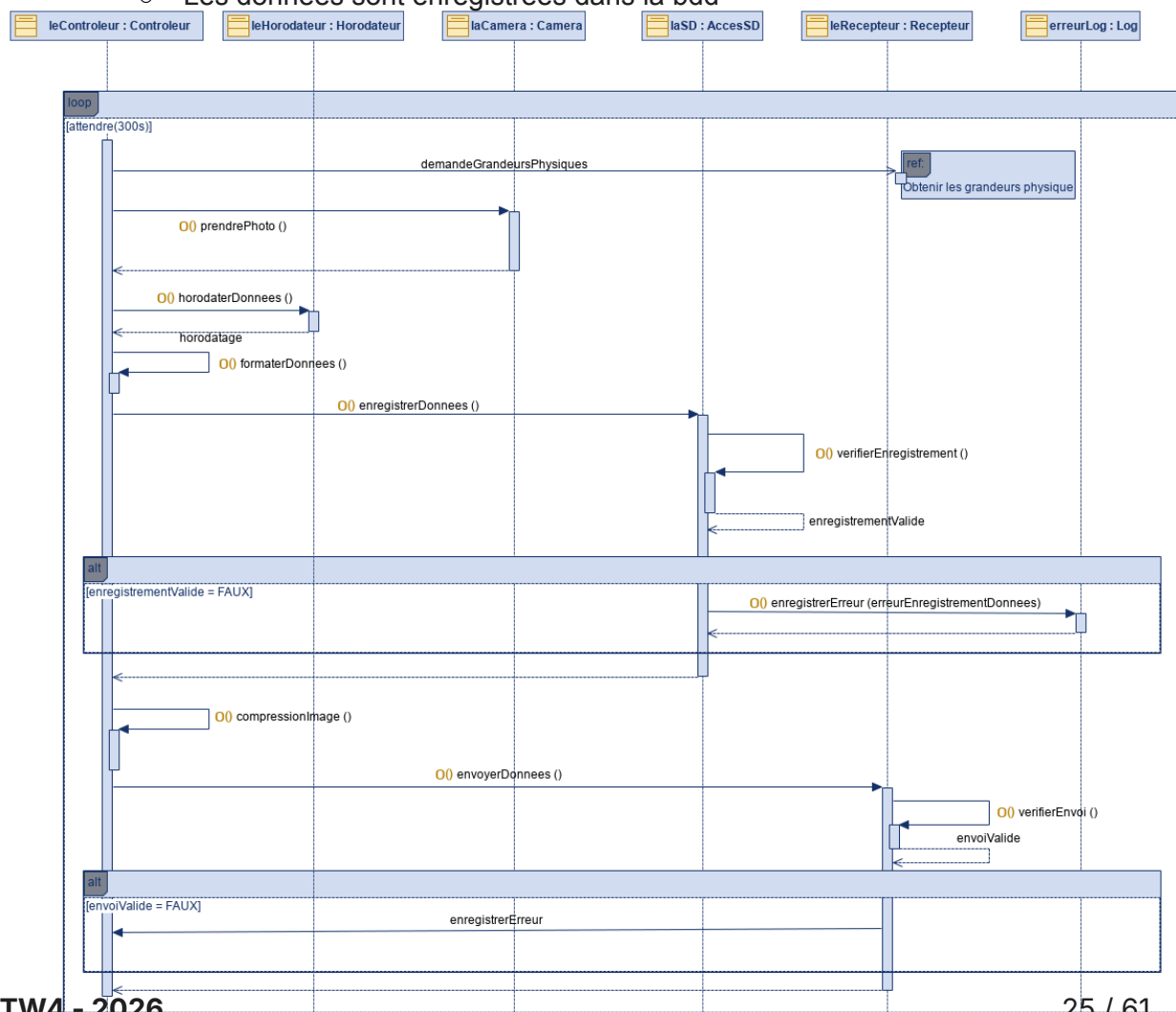
- **Acteur :**
 - Ballon
- **Classes :**
 - Récepteur
 - Modèle
 - Log
- **Pré-conditions :**
 - La système sous tension et le script de réception des données est actif
 - La ballon est en cours d'émission
- **Scénario Principal :**
 - Le Récepteur reçoit la trame du ballon, son intégrité est vérifiée ainsi que la présence de chaque donnée individuellement. Puis les données sont enregistrées dans la bdd
- **Scénario d'Erreur 1:**
 - Si la trame est invalide , alors enregistrer dans le fichier log que celle ci est corrompue.
- **Scénario d'Erreur 2 :**
 - Si la donnée reçue est absente alors enregistrer "null" pour cette ligne
- **Post-condition :**
 - Les données sont enregistrées dans la base de données



Cas d'Utilisation N°3 : Émettre la télémétrie [NACELLE]

Description : Émettre la télémétrie.

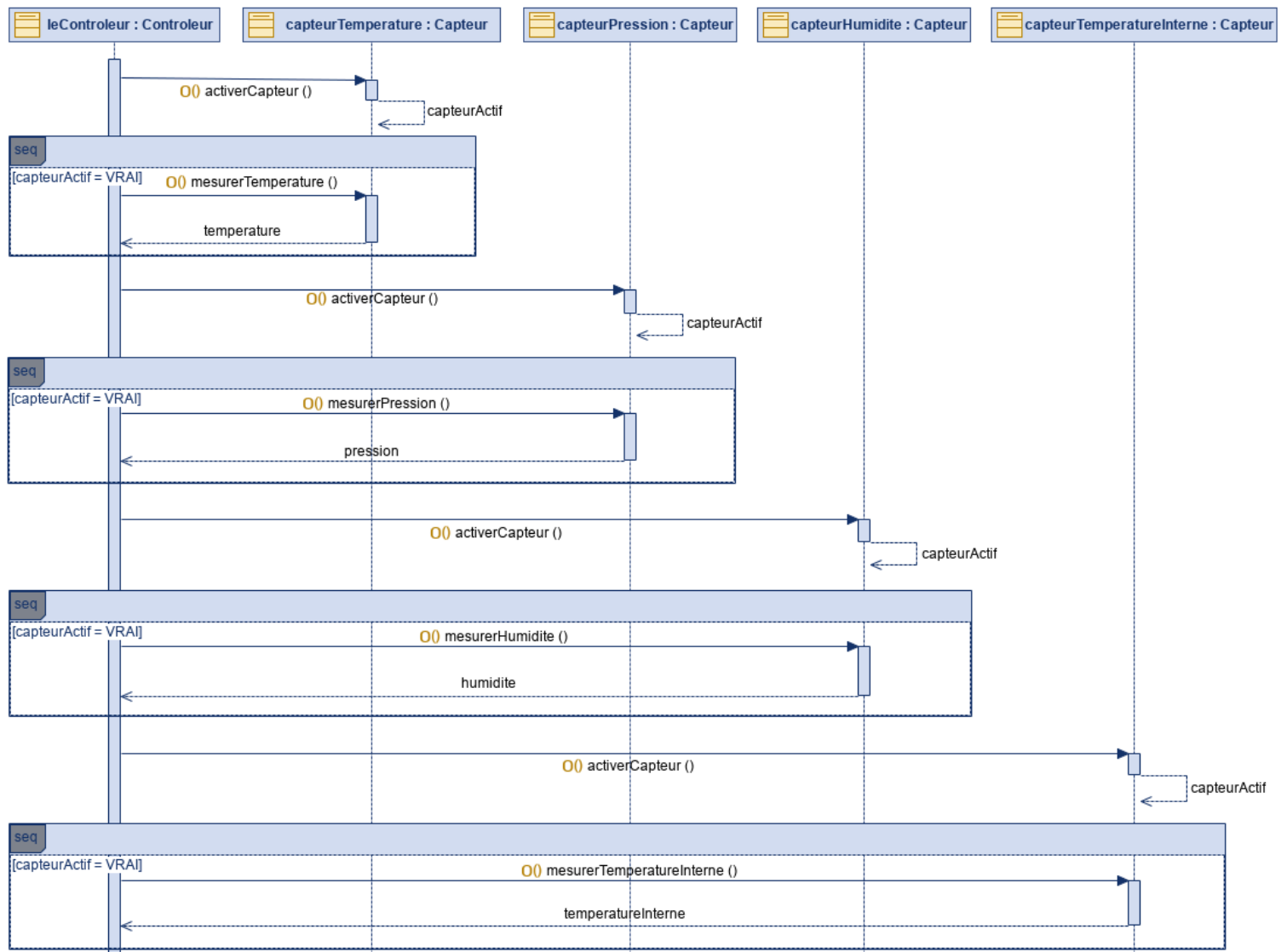
- **Classes :**
 - Contrôleur
 - Caméra
 - Carte SD
 - Récepteur
 - Log
 - Horodateur
- **Pré-conditions :**
 - La système sous tension et le script de réception des données est actif
 - La ballon est en cours d'émission
- **Scénario Principal :**
 - Le contrôleur demande les grandeurs physiques , prends une photo avant de horodater puis formater les données, Ensuite celles-ci sont enregistrées sur la carte SD . L'image est ensuite compressée avant que le tout soit envoyé.
- **Scénario d'Erreur 1 :**
 - Si l'enregistrement sur la carte SD n'a pas fonctionné, alors mettre un message dans les log.
- **Scénario d'Erreur 2 :**
 - Si l'envoi n'a pas réussi alors enregistrer un message dans les logs
- **Post-condition :**
 - Les données sont enregistrées dans la bdd



Cas d'Utilisation N°4 : Obtenir les grandeurs physiques [NACELLE]

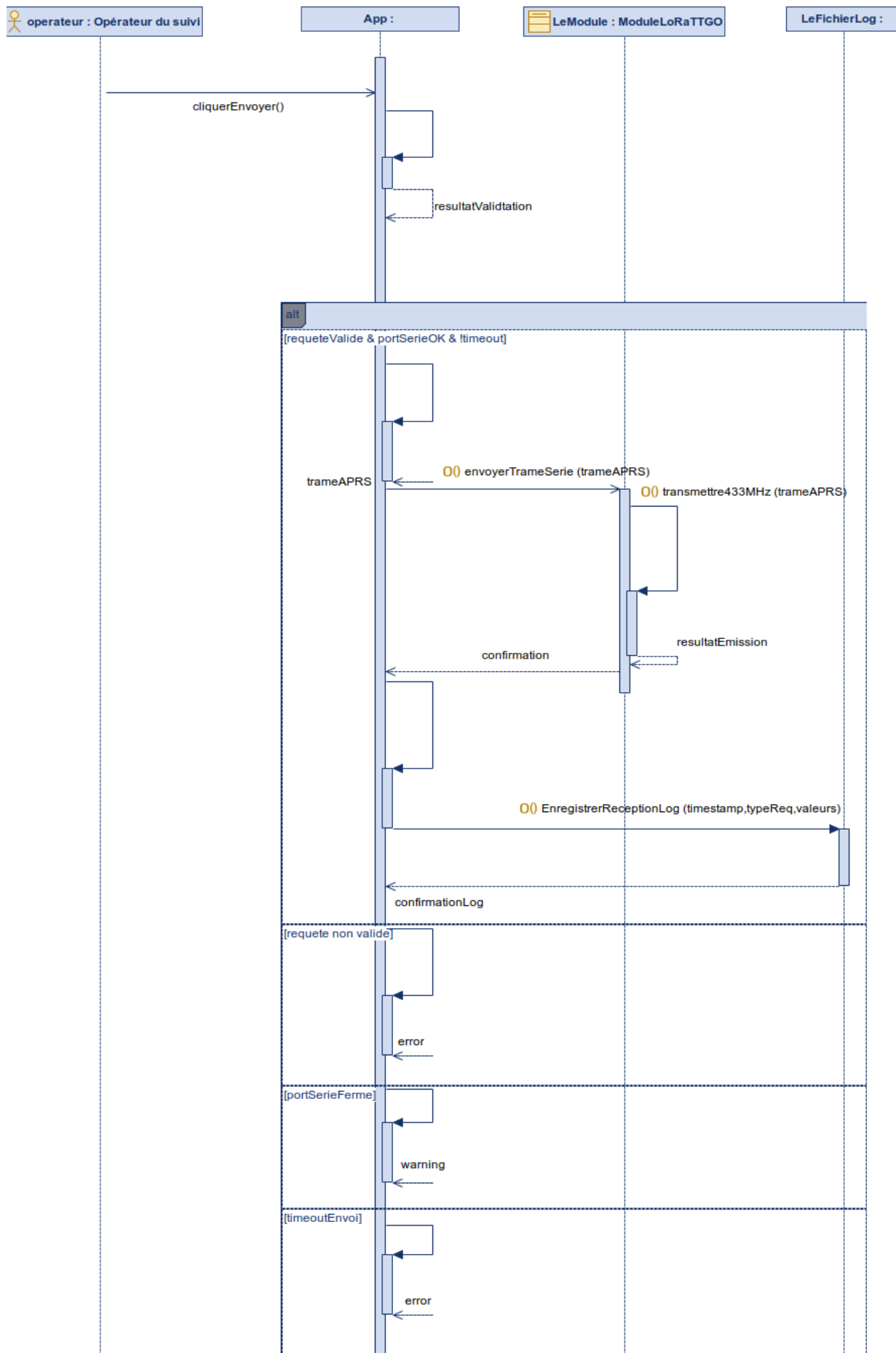
Description : Obtenir les grandeurs physiques.

- **Classes :**
 - Contrôleur
 - Capteur de Température
 - Capteur de Pression
 - Capteur d'Humidité
 - Capteur de Température Interne
- **Pré-conditions :**
 - La Raspberry du ballon est initialisée
 - Les capteurs sont alimentés et connectés au bus de communication
 - Les capteurs sont opérationnels
- **Scénario Principal :**
 - Pour chaque donnée, activer le capteur puis récupérer la valeur mesurée



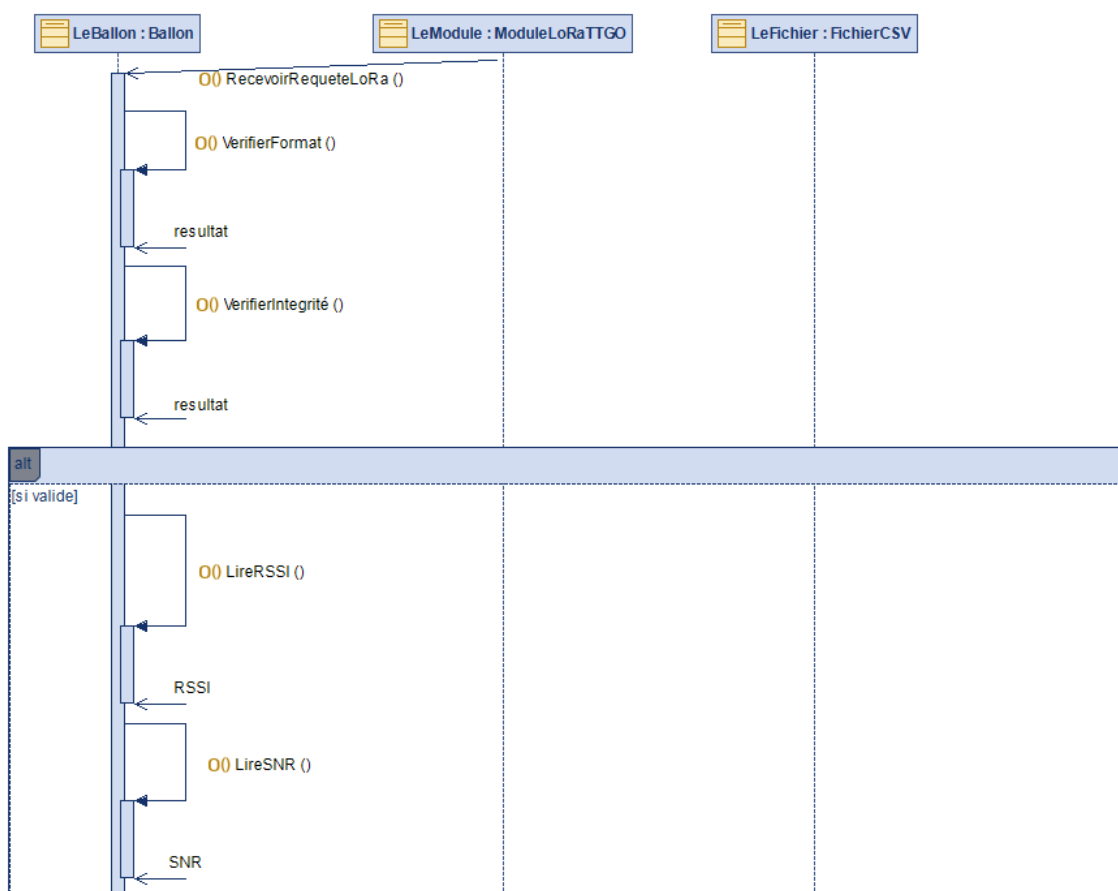
Cas d'Utilisation N°1 : Envoyer une requête [Sol]

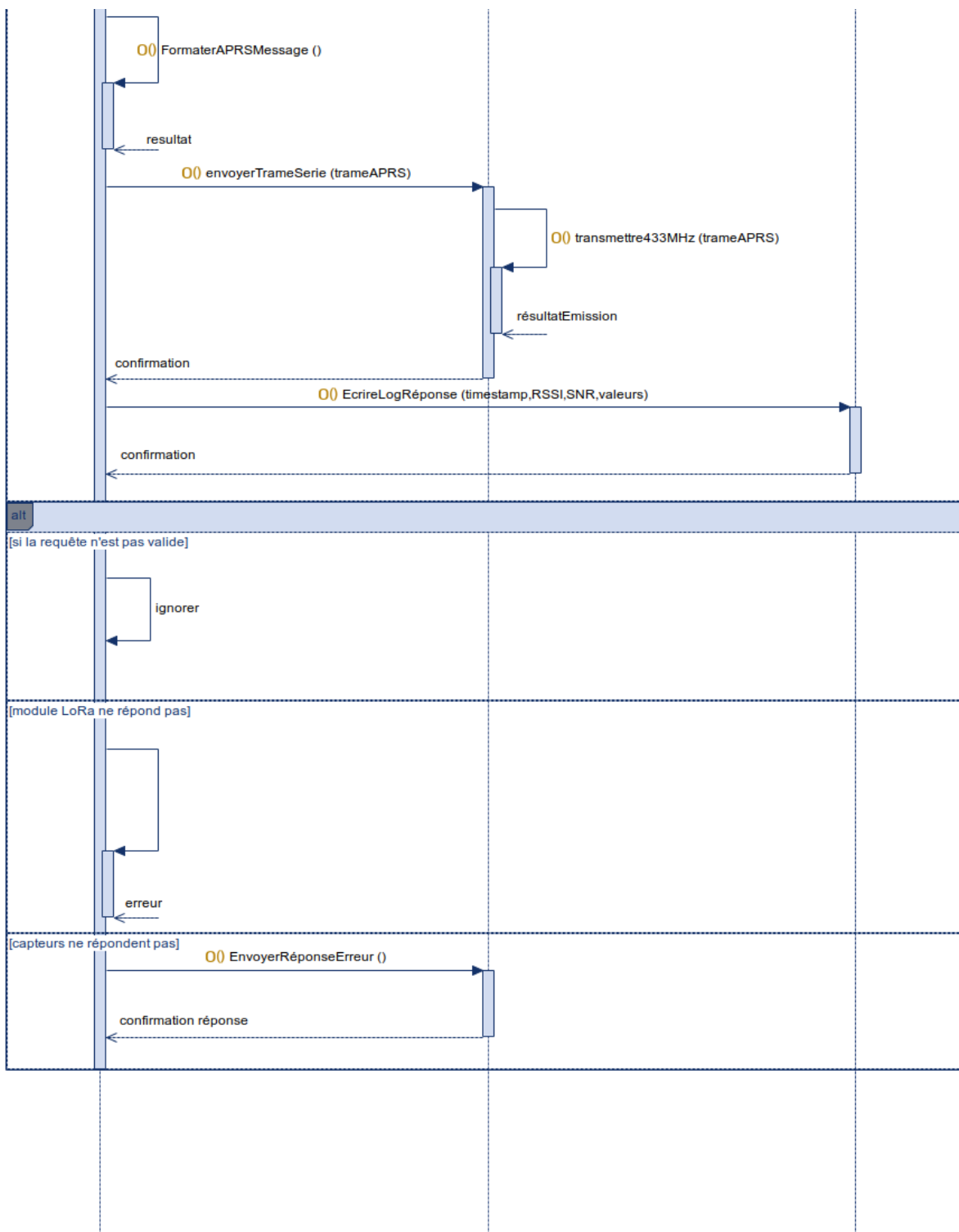
- **Description :** Envoyer une requête à la nacelle.
- **Acteurs :**
 - Opérateur du suivi
 - App
 - LeModule
 - LeFichierLog
- **Pré-conditions :**
 - La liaison LoRa montante est opérationnelle
 - L'application UI est lancée sur le PC
 - La nacelle est en vol et à portée radio
 - L'opérateur a accès à l'interface de saisie
- **Scénario principal :**
 - L'utilisateur choisit sa requête sur une liste déroulante et envoie une requête via la gateway LoRa Lycée.
- **Scénario Erreur 1 :**
 - Si la requête est non valide, le module ignore la requête.
- **Scénario Erreur 2 :**
 - Si le port série est fermé ou rencontre tout autre type d'erreur, alors on redémarre le port.
- **Scénario Erreur 3 :**
 - Si la requête est timeout, un avertissement est envoyé sur l'IHM
- **Post-conditions :**
 - La requête est formatée selon le protocole APRS Message
 - La requête est transmise au module LoRa client
 - Un log de la requête est enregistré
 - Un message de confirmation est affiché à l'écran



Cas d'Utilisation N°2 : Répondre à la requête

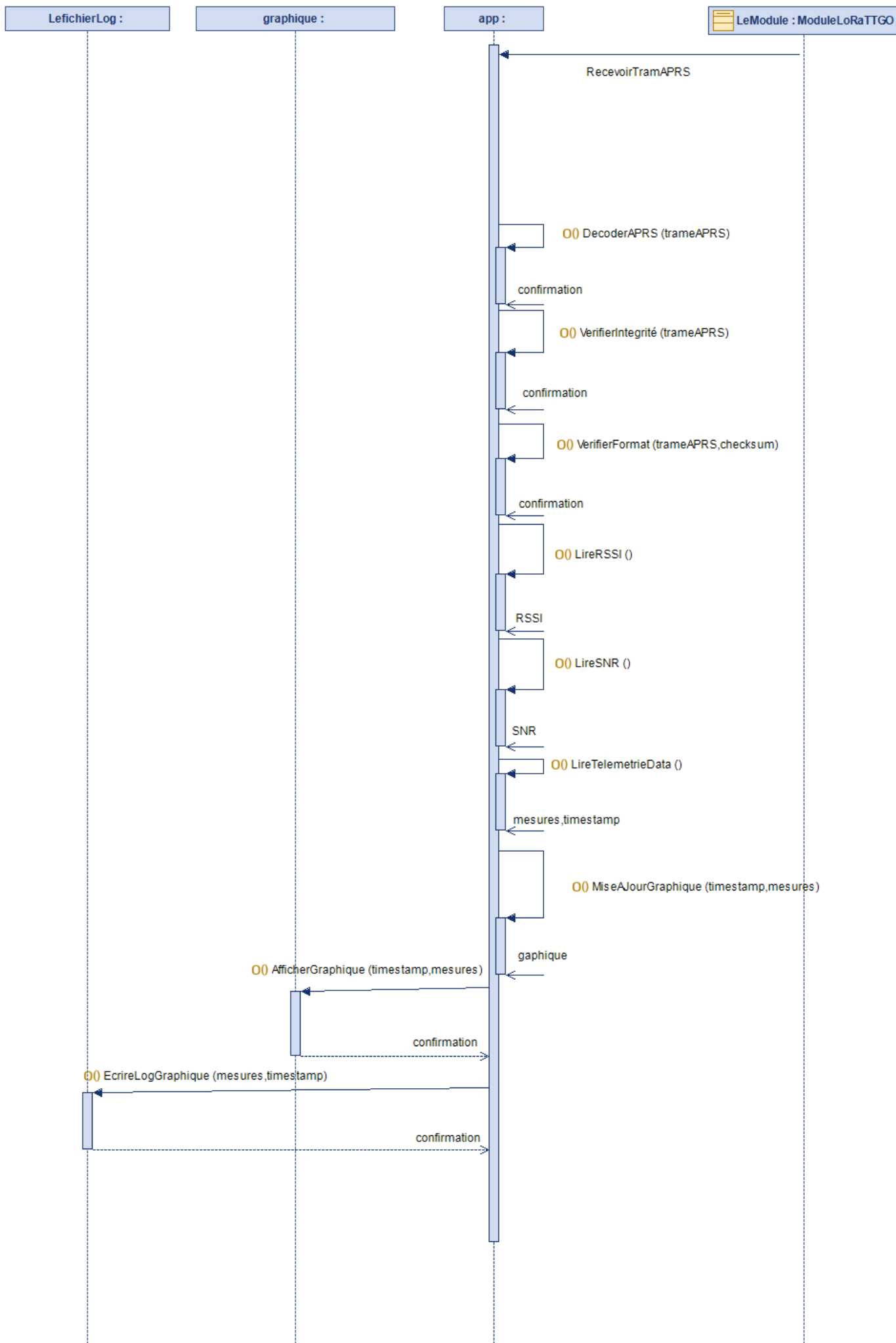
- **Description :** Répondre à une requête en vol
- **Acteurs :**
 - Ballon
 - ModuleLoRaTTGO
 - FichierCSV
- **Pré-conditions :**
 - La liaison LoRa montante est opérationnelle
 - L'application UI est lancée sur le PC
 - La nacelle est en vol et à portée radio
 - L'opérateur a accès à l'interface de saisie
- **Scénario principal :**
 - Le Ballon lit la requête montante, la traite puis répond à la requête en envoyant une trame descendante à la station au sol.
- **Scénario Erreur 1 :**
 - Si la requête est non valide, le module ignore la requête.
- **Scénario Erreur 2 :**
 - Si le module LoRa ne répond pas, une erreur interne est envoyée et on redémarre le capteur.
- **Scénario Erreur 3 :**
 - Si les capteurs ne répondent pas, une erreur interne est envoyée et on redémarre les capteurs
- **Post-conditions :**
 - La requête est formatée selon le protocole défini
 - La requête est transmise au module LoRa client
 - Un log de la requête est enregistré
 - Un message de confirmation est affiché à l'écran





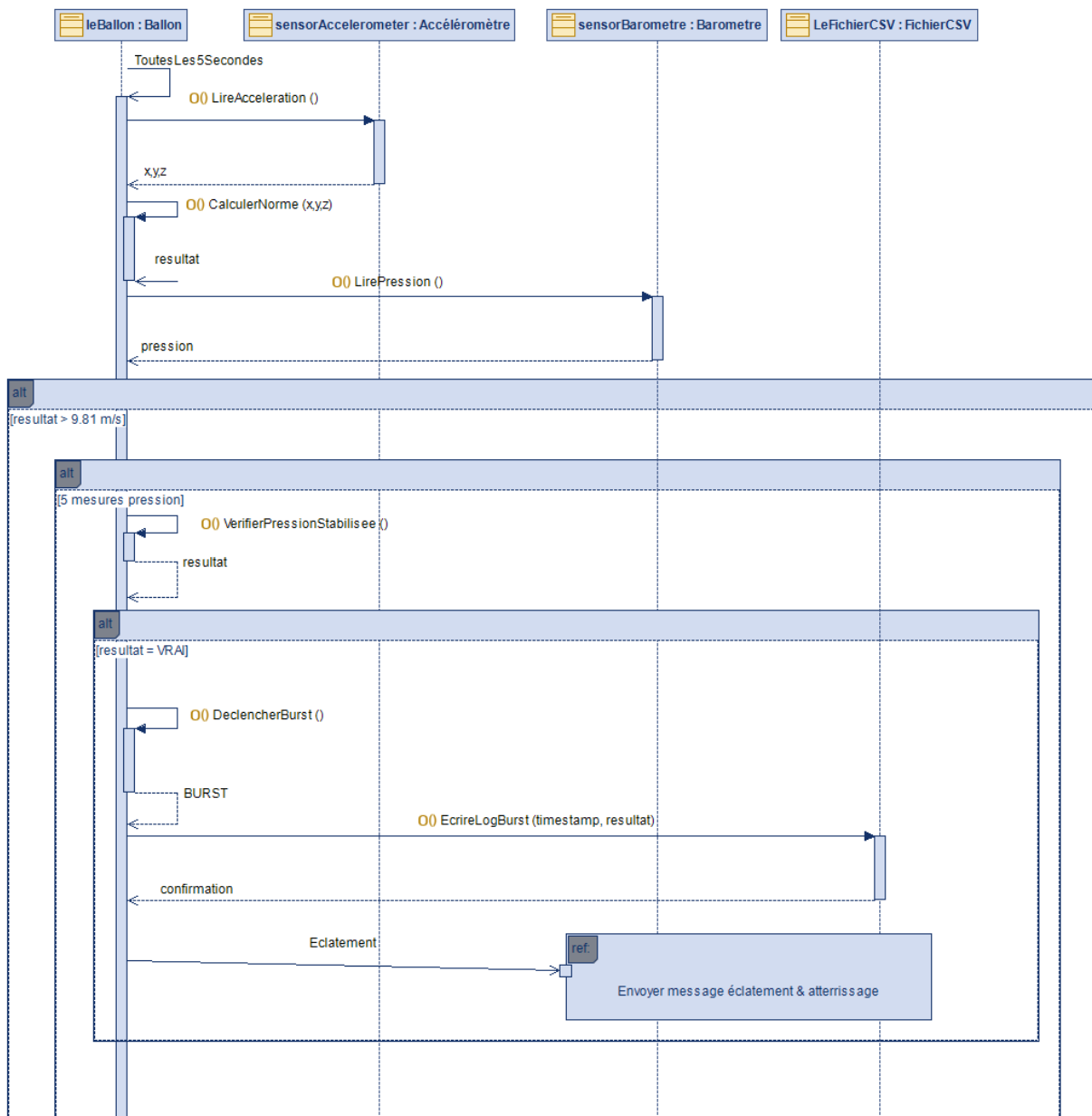
Cas d'Utilisation N°3 : Lire et afficher la réponse

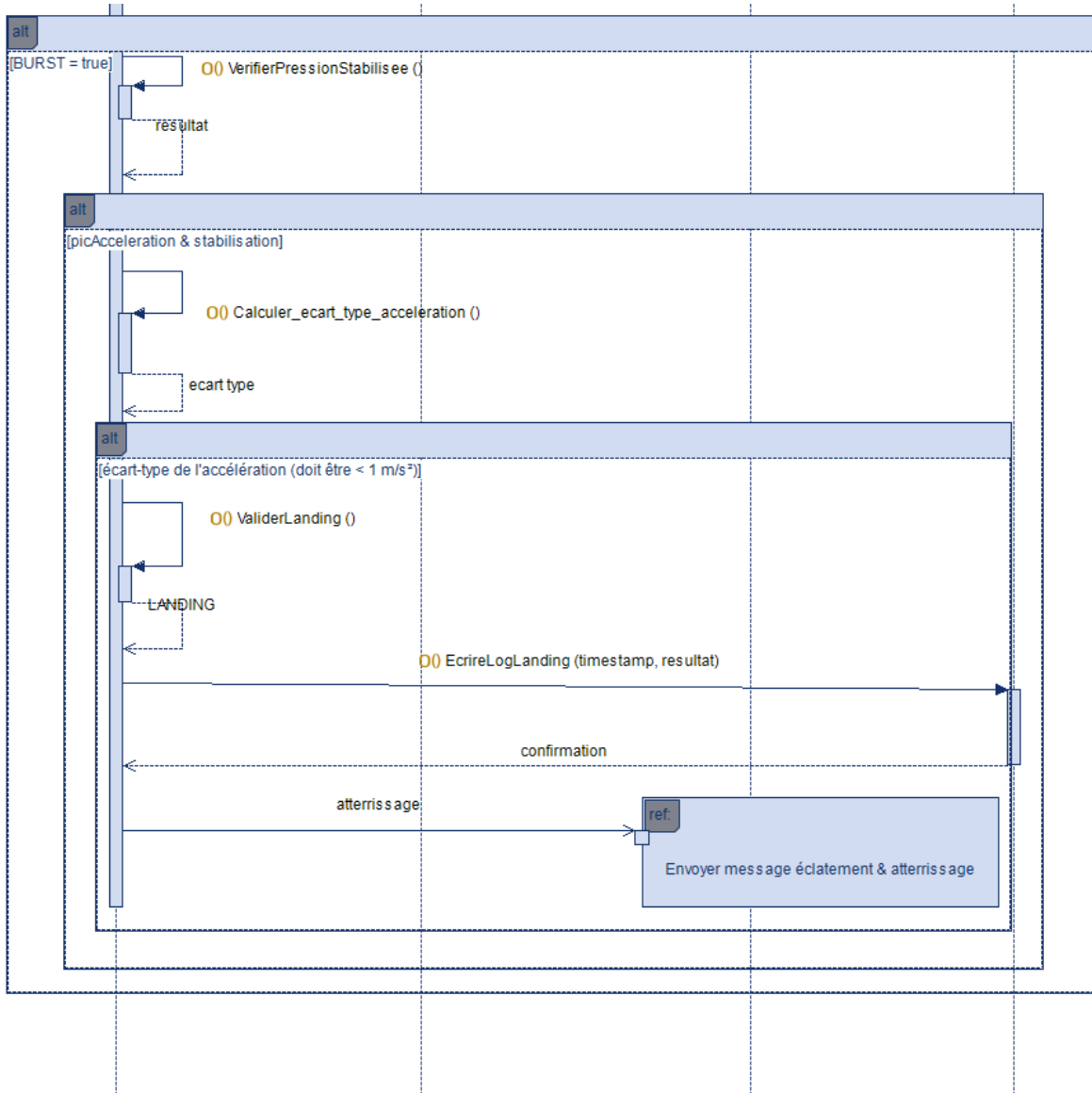
- **Description :** Lire et afficher la réponse de la nacelle
- **Acteurs :**
 - FichierLog
 - graphique
 - app
- **Pré-conditions :**
 - Une requête a été envoyée précédemment
 - La liaison LoRa descendante est opérationnelle
 - L'application UI est en cours d'exécution
 - Le timeout d'attente n'est pas dépassé
- **Scénario principal :**
 - L'application QT lit la réponse de la requête du ballon via le module de réception et d'émission de requête LoRa.
- **Post-conditions :**
 - La réponse est décodée et validée
 - Les données sont affichées sur l'interface
 - L'historique est mis à jour
 - Un log de réception est enregistré
 - Les données sont sauvegardées pour analyse ultérieure



Cas d'Utilisation N°4 : Détecter éclatement & atterrissage

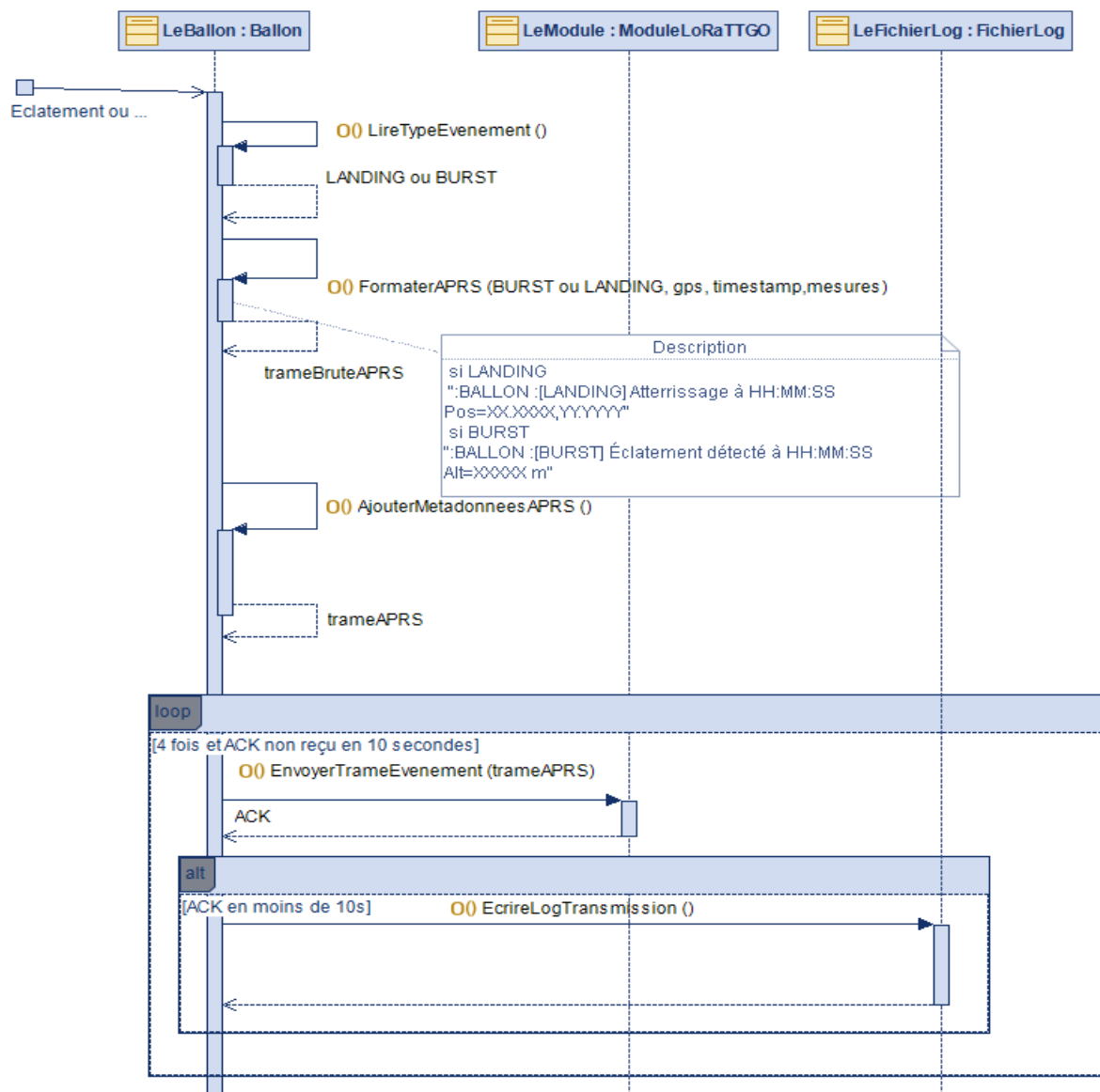
- **Description** : Détecter éclatement & atterrissage de la nacelle
- **Acteurs** :
 - FichierLog
 - graphique
 - app
- **Pré-conditions** :
 - La nacelle est en vol
 - L'accéléromètre est calibré et opérationnel
 - Le baromètre fonctionne correctement
 - Le système de mesure tourne en continu
- **Scénario principal** :
 - Le Ballon demande aux capteurs toutes les 5 secondes si l'accélération a brusquement changé puis calcule la vitesse actuelle pour en déduire l'état du ballon entre : en vol, BURST, LANDING.
- **Post-conditions** :
 - L'événement est identifié avec précision
 - Un flag interne est activé (BURST ou LANDING)
 - L'événement est horodaté
 - Les données sont enregistrées dans le CSV
 - Le module d'envoi de message est notifié





Cas d'Utilisation N°5 : Envoyer message éclatement & atterrissage

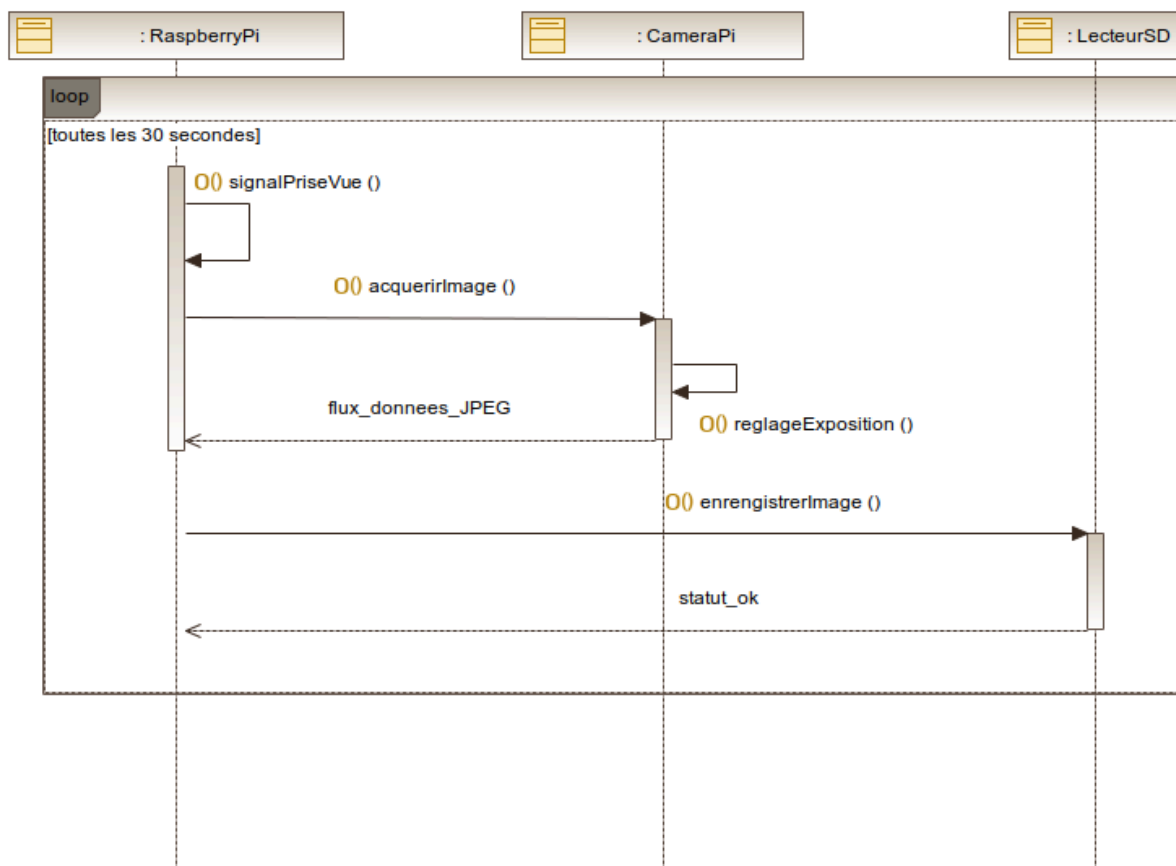
- **Description :** Envoyer message d'événement critique via LoRa
- **Acteurs :**
 - Ballon
 - ModuleLoRa
 - FichierLog
- **Pré-conditions :**
 - Un événement a été détecté (flag BURST ou LANDING = true)
 - Le module LoRa RA-02 est opérationnel
 - La liaison descendante est disponible
 - Le système a accès aux coordonnées GPS actuelles
- **Scénario principal :**
 - Suivant l'état actuelle du Ballon et en continuité du diagramme de séquence Détecter éclatement & atterrissage, le ballon formate un message sous le format APRS Message pour le distribué au module LoRa et l'envoyer à la station au sol.
- **Post-conditions :**
 - Le message APRS est formaté correctement
 - Le message est transmis sur 433 MHz
 - La transmission est confirmée
 - Un log de transmission est créé
 - Le système reprend le cycle normal de transmission



Cas d'Utilisation N°1 : Prendre une photo du ballon [NACELLE]

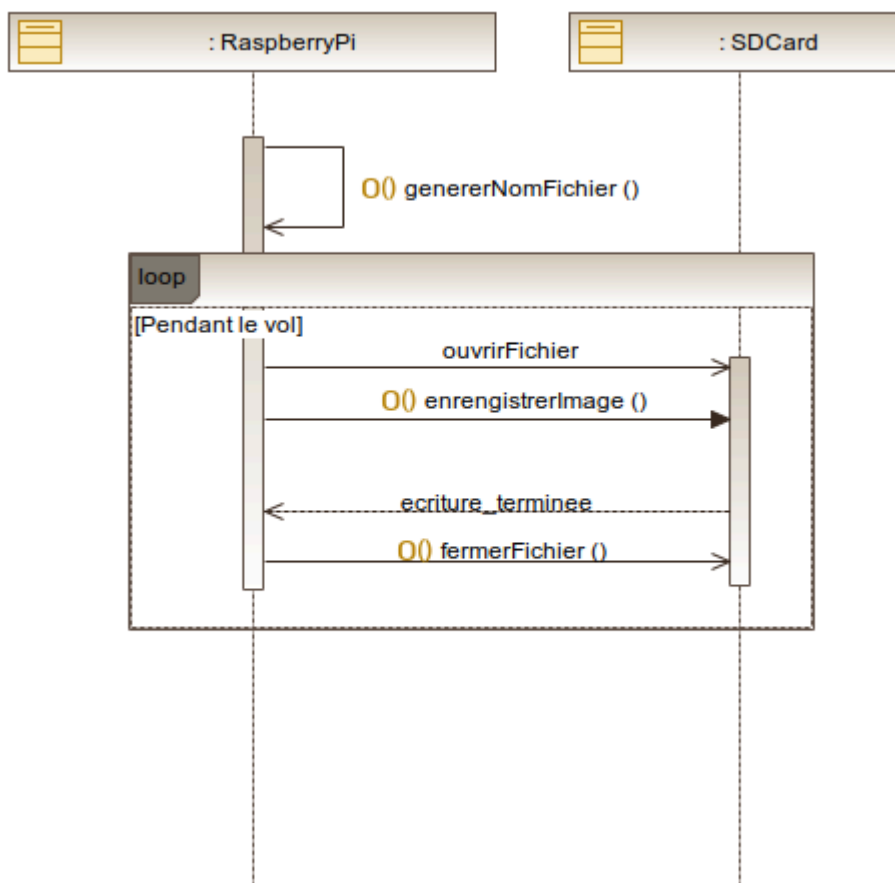
Description : Prendre une photo du ballon.

- **Classe(s) :**
 - RaspberryPi
 - CameraPi
 - LecteurSD
- **Pré-conditions :**
 - La Raspberry est alimentée et la caméra est initialisée
- **Scénario Principal :**
 - Le script de capture se déclenche (selon un timer).
 - Le système envoie une commande à la caméra.
 - La caméra capture l'image et la transmet au processeur.
 - Le système convertit le flux en fichier image.
- **Scénario Alternatif :**
 - Si la caméra est déconnectée, le système enregistre une erreur
- **Post-condition :**
 - Un fichier image est disponible dans la mémoire temporaire du système



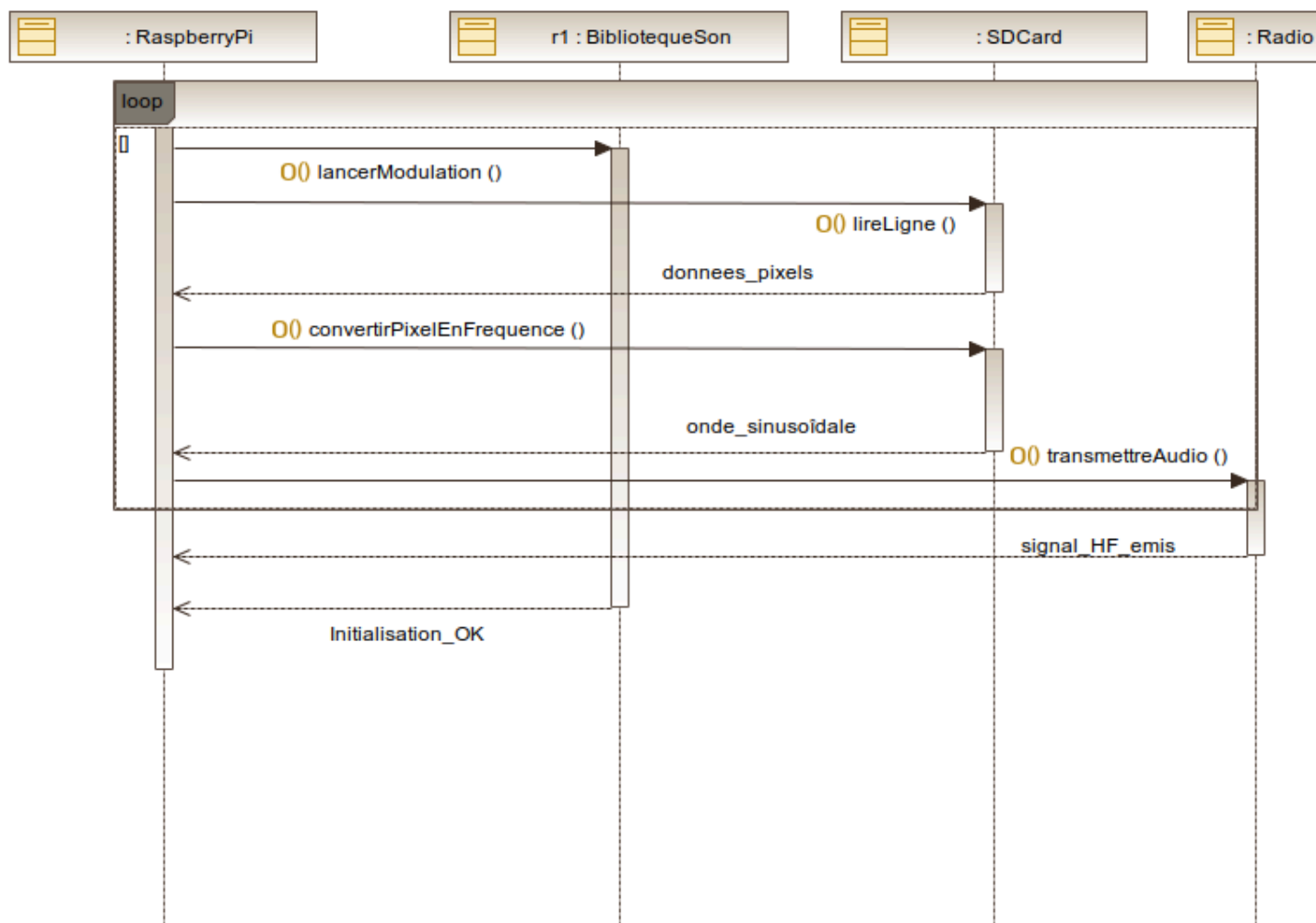
Cas d'Utilisation N°2 : Enregistrer la photo sur la carte SD [NACELLE]

- **Description** : Enregistrer la photo via la Raspberry sur la carte SD
- **Classe(s)** :
 - RaspberryPi
 - SDCard
- **Pré-conditions** :
 - La caméra a capturé une image; la carte SD est montée
- **Scénario principale** :
 - Le système ouvre un nouveau fichier avec un nom unique
 - Le système écrit les données de l'image sur la carte SD.
 - Le système ferme le fichier et vérifie que l'écriture est terminée.
- **Scénario Alternatif** :
 - Si la caméra est déconnectée, le système enregistre une erreur
- **Post-condition** :
 - Un nouveau fichier .jpg est créé sur la carte SD



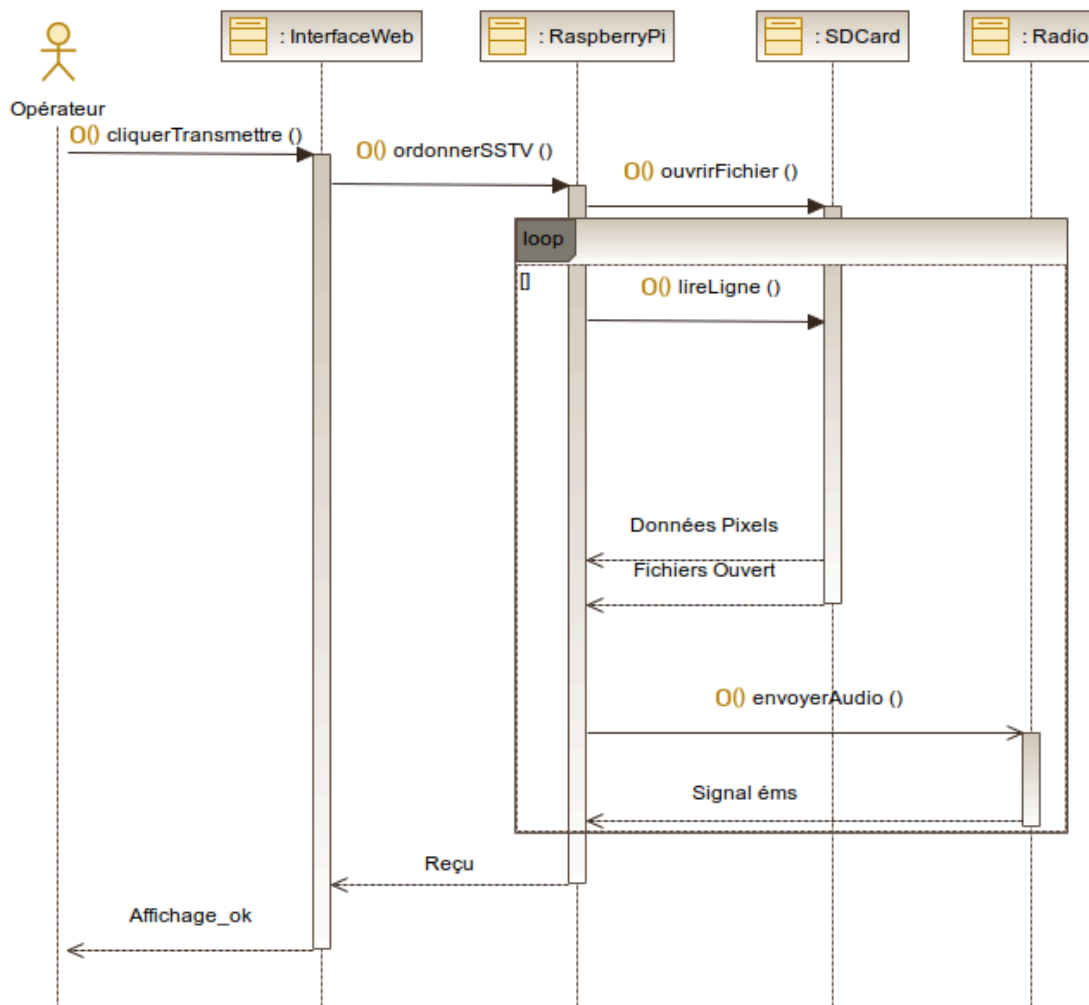
Cas d'Utilisation N°3 : Moduler le signal HF [NACELLE]

- **Description** : moduler le signal haute fréquence avec la Raspberry Pi via la carte SD et l'émetteur.
- **Classe(s)** :
 - RaspberryPi
 - BibliothequeSon
 - SDCard
 - Radio
- **Scénario principal** :
 - Le système ouvre un nouveau fichier avec un nom unique.
 - Le système écrit les données de l'image sur la carte SD.
 - Le système ferme le fichier et vérifie que l'écriture est terminée.
- **Scénario alternatif** :
 - Si la carte SD est pleine, le système supprime la plus ancienne photo ou arrête l'enregistrement.
- **Pré-conditions** :
 - Le fichier image est disponible ; les paramètres de transmission (SSTV Martin 1) sont définis.
- **Post-condition** :
 - Un signal modulé est prêt à être envoyé à l'émetteur



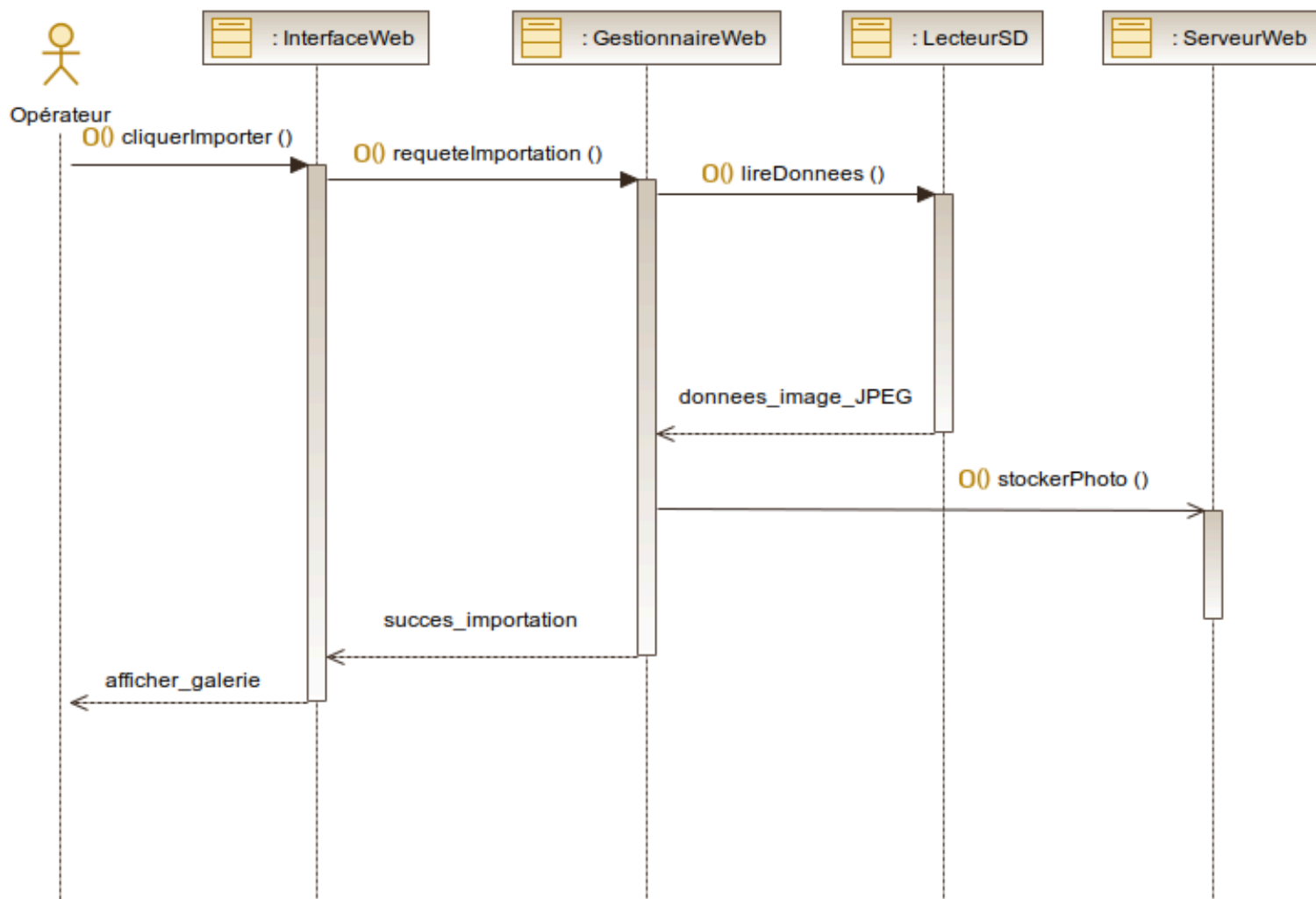
Cas d'Utilisation N°4 : Transmettre la photo au sol [SOL]

- **Description** : Prendre une photo via la caméra et qui sera enregistrer sur la carte SD.
- **Acteur(s)** :
 - Opérateur
- **Classe(s)** :
 - InterfaceWeb
 - RaspberryPi
 - SDCard
 - Radio
- **Scénario principal** :
 - Le système embarqué (Ballon) fragmente l'image pour la transmission.
 - Le système module les données en signal audio (SSTV).
 - L'émetteur radio diffuse le signal vers la terre.
- **Scénario alternatif** :
 - Si la carte SD est pleine, le système supprime la plus ancienne photo ou arrête l'enregistrement.
- **Pré-condition** :
 - Une photo est prête à être envoyée ; la liaison radio est établie.
- **Post-condition** :
 - Le flux de données est diffusé sur les ondes HF.



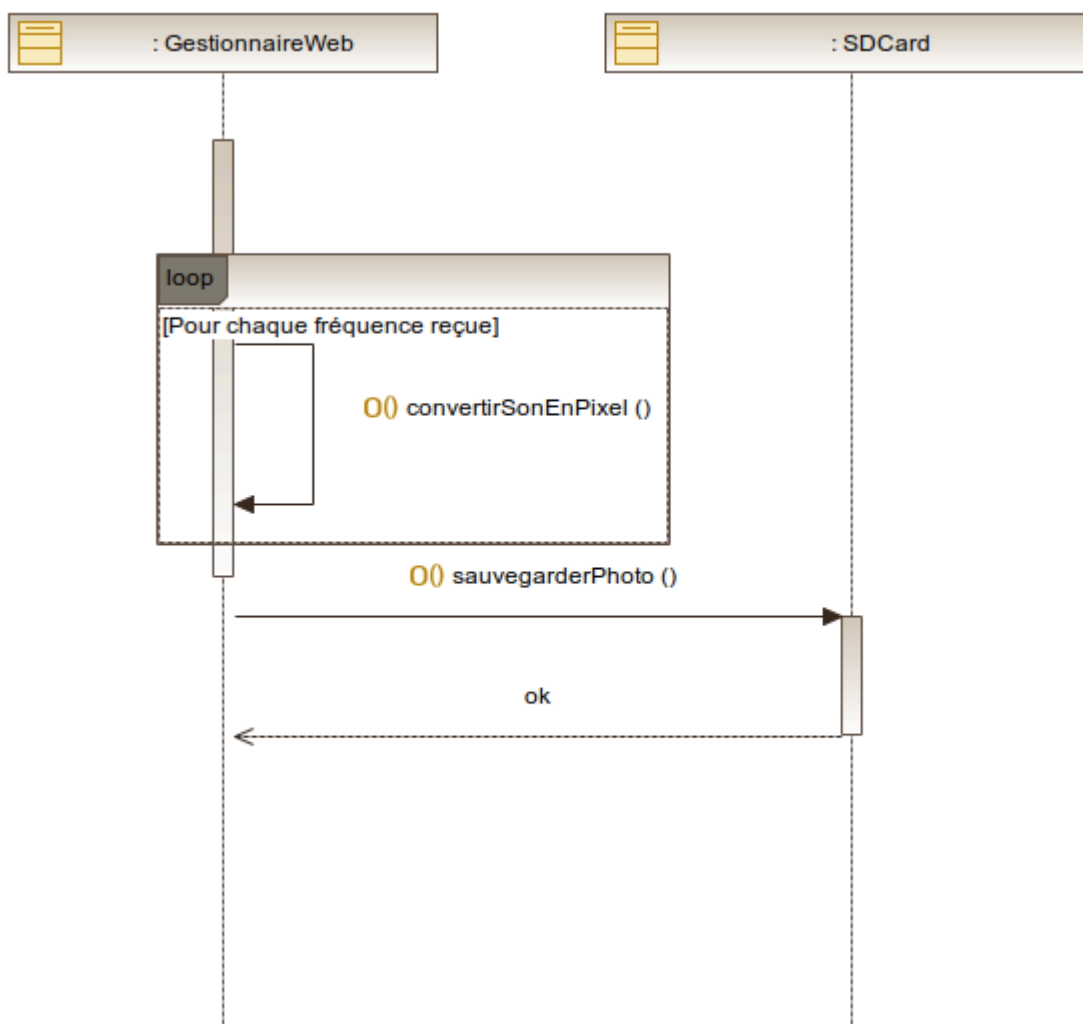
Cas d'Utilisation N°5 : Importer les photos enregistrées [SOL]

- **Description** : Importer les photos et les mettre dans le serveur Web.
- **Acteur(s)** :
 - Opérateur
- **Classe(s)** :
 - InterfaceWeb
 - GestionnaireWeb
 - LecteurSD
 - ServeurWeb
- **Scénario principal** :
 - L'opérateur insère la carte SD dans son ordinateur.
 - L'opérateur sélectionne le dossier contenant les photos du vol.
 - L'opérateur lance la copie des fichiers vers le stockage local ou le serveur web.
- **Scénario alternatif** :
 - Si la carte SD est illisible ou endommagée, l'importation est impossible.
- **Pré-conditions** :
 - Le ballon a été récupéré ; la carte SD est extraite de la Raspberry Pi.
- **Post-condition** :
 - Les photos sont transférées sur l'ordinateur de traitement ou le serveur.



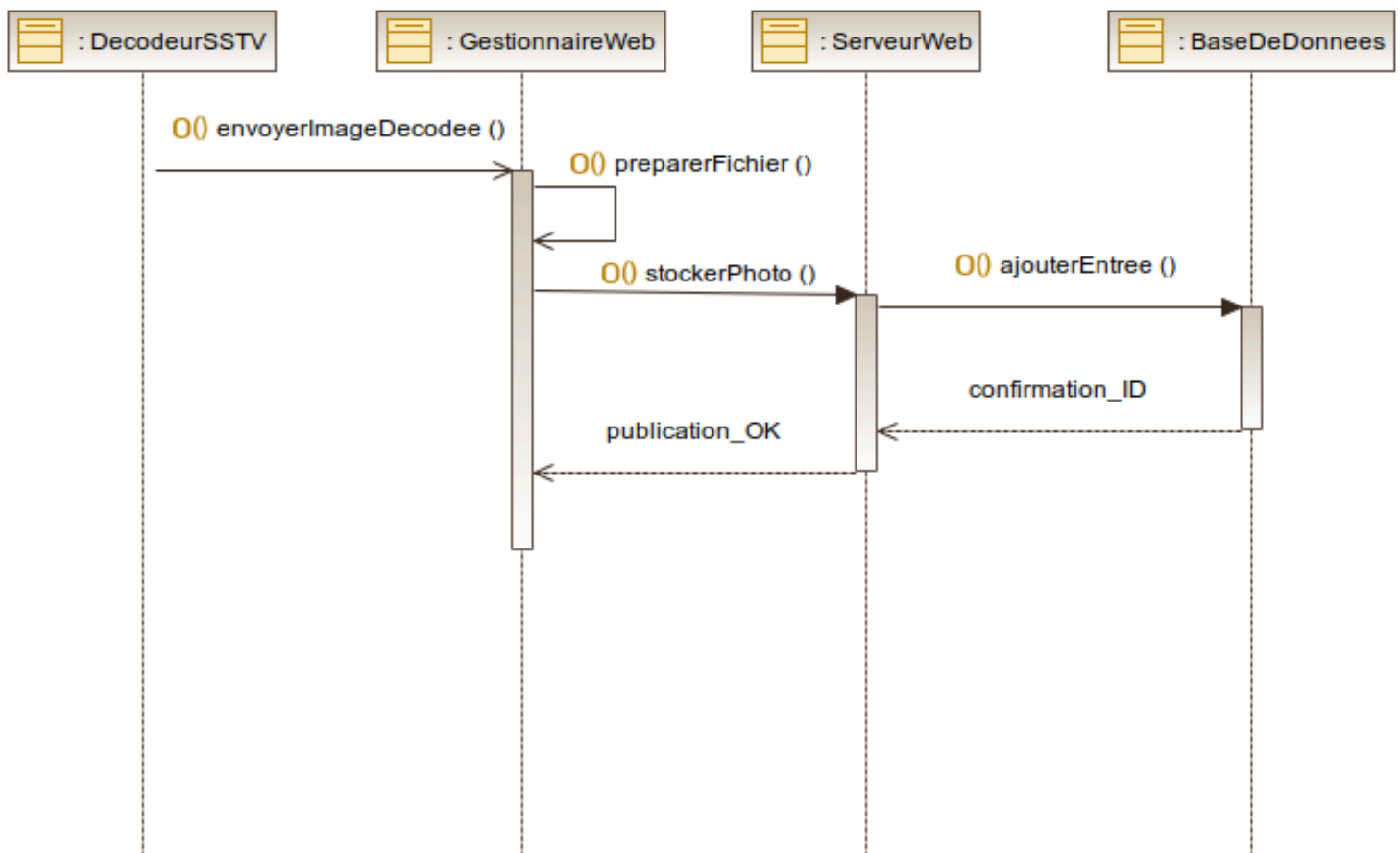
Cas d'Utilisation N°6 : Décoder le signal HF [SOL]

- **Description** : Décoder le signal haute fréquence via le gestionnaire.
- **Classe(s)** :
 - GestionnaireWeb
 - SDCard
- **Scénario principal** :
 - La station au sol reçoit le flux audio provenant de la radio.
 - Le logiciel analyse les fréquences du signal (SSTV).
 - Le système reconstruit l'image ligne par ligne.
 - Le système enregistre l'image décodée dans le dossier de destination du serveur.
- **Scénario alternatif** :
 - Si le signal est trop faible, l'image n'est pas enregistrée ou reste partielle.
- **Pré-conditions** :
 - Le récepteur radio est accordé sur la bonne fréquence et reçoit un signal.
- **Post-condition** :
 - Le signal audio est transformé en fichier image lisible.



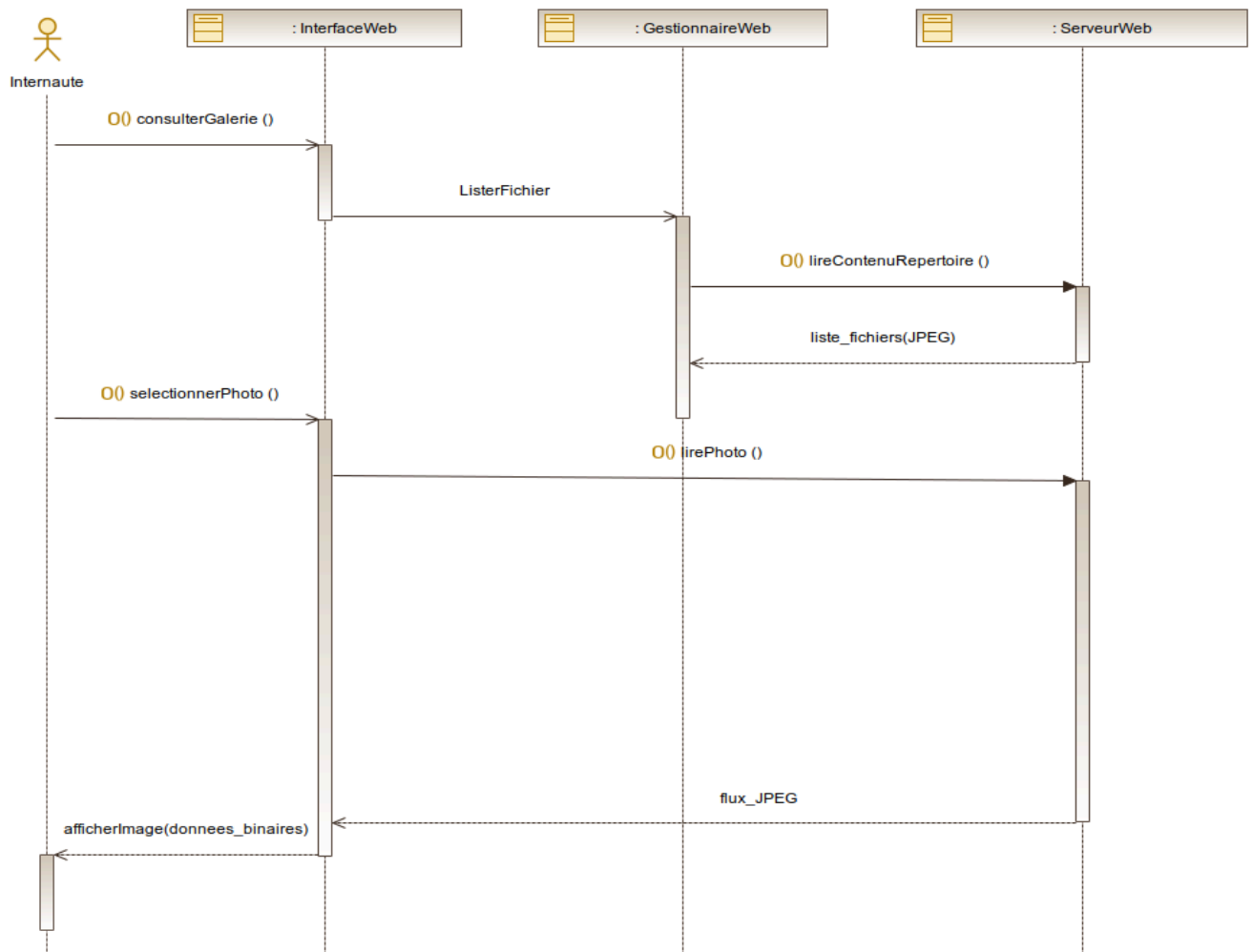
Cas d'Utilisation N°7 : Enregistrer une photo [SOL]

- **Description** : Enregistrer une photo via le gestionnaire, le serveur web et la base de données
- **Classe(s)** :
 - DecodeurSSTV
 - GestionnaireWeb
 - ServeurWeb
 - BaseDeDonnees
- **Scénario principal** :
 - L'internaute fait un clic droit ou appuie sur le bouton "Enregistrer/Télécharger".
 - Le serveur web autorise le transfert du fichier.
 - Le navigateur enregistre l'image dans le dossier "Téléchargements" de l'utilisateur.
- **Scénario alternatif** :
 - Si le serveur est déconnecté, le téléchargement échoue.
- **Pré-condition** :
 - L'internaute est sur l'interface web et une photo est affichée.
- **Post-condition** :
 - La photo est téléchargée sur l'appareil de l'internaute.



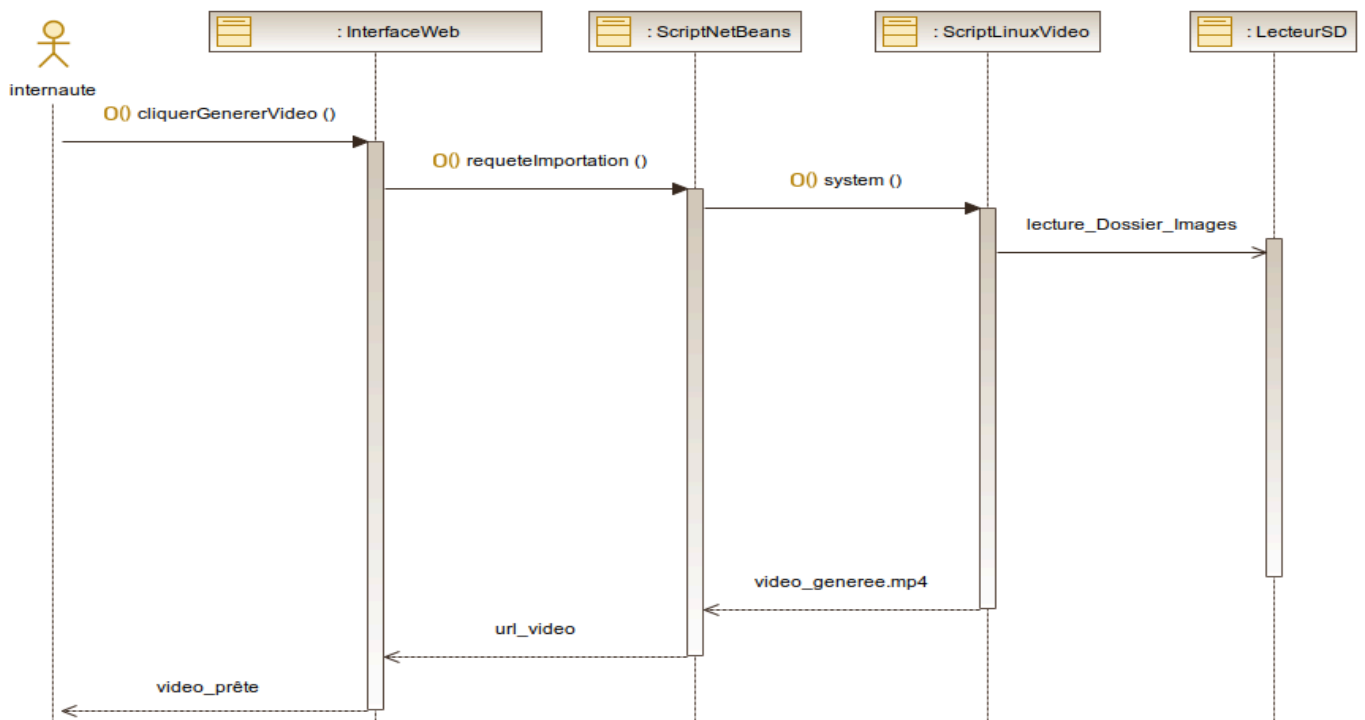
Cas d'Utilisation N°8 : Visualiser une photo [SOL]

- **Description :** Visualiser une photo via l'interface Web
- **Classes(s) :**
 - InterfaceWeb
 - GestionnaireWeb
 - ServeurWeb
- **Scénario principal :**
 - L'internaute saisit l'adresse IP ou l'URL du serveur web dans son navigateur.
 - L'internaute accède à la page de visualisation (galerie ou page d'accueil).
 - L'internaute sélectionne la photo qu'il souhaite voir (ou la page affiche automatiquement la dernière photo reçue).
 - Le serveur web envoie le fichier image au navigateur.
 - Le navigateur affiche l'image avec ses informations (heure de réception, etc.).
- **Scénario alternatif :**
 - Si le serveur est inaccessible, le navigateur affiche une erreur "404" ou "Délai dépassé".
 - Si aucune photo n'est présente sur le serveur, un message "Aucune image reçue pour le moment" s'affiche à la place de la photo.
- **Pré-condition :**
 - Le serveur Web (sur la station au sol) est actif. Des photos ont été décodées et stockées dans le dossier accessible par le serveur.
- **Post-condition :**
 - L'internaute visualise l'image sélectionnée sur son navigateur.



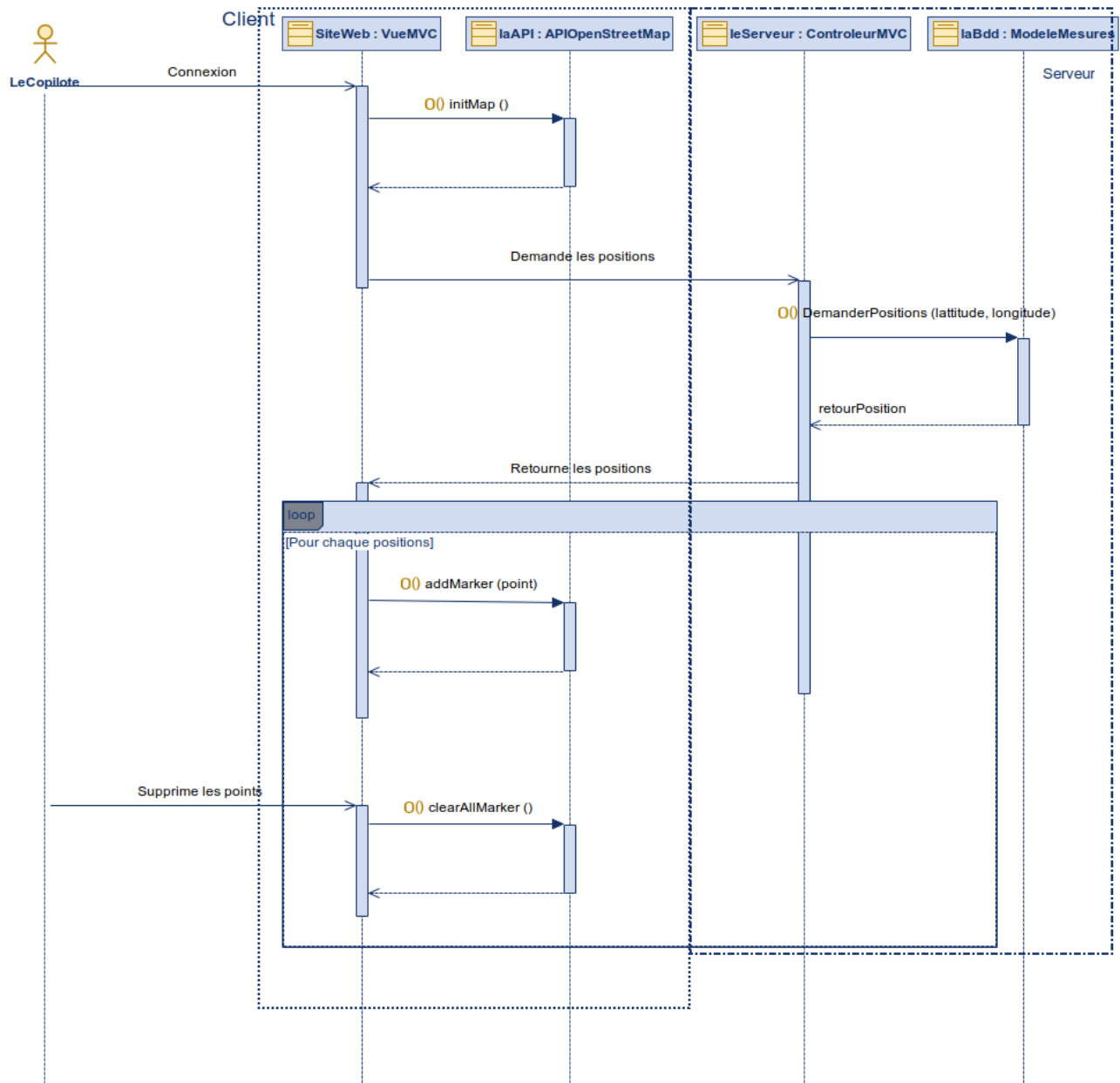
Cas d'Utilisation N°9 : Générer une vidéo à partir des photos [SOL]

- **Description** : Générer une vidéo via une ligne de commande linux et de photos stockées
- **Acteur(s)** :
 - Internaute
- **Classes(s)** :
 - InterfaceWeb
 - ScriptNetBeans
 - ScriptLinuxVideo
 - LecteurSD
- **Scénario principal** :
 - L'internaute clique sur le bouton "Générer la vidéo" sur l'interface web.
 - Le serveur web lance le script de traitement d'images.
 - Le système assemble les photos par ordre chronologique.
 - Le système génère le fichier vidéo final.
 - L'interface affiche la vidéo à l'internaute.
- **Scénario alternatif** :
 - Si le dossier de photos est vide, le système affiche un message d'alerte.
- **Pré-condition** :
 - Les photos du vol sont présentes sur le serveur web.
- **Post-condition** :
 - Un fichier vidéo est créé et prêt à être visionné.



Cas d'Utilisation N°1 : Visualiser la position du véhicule par rapport au ballon

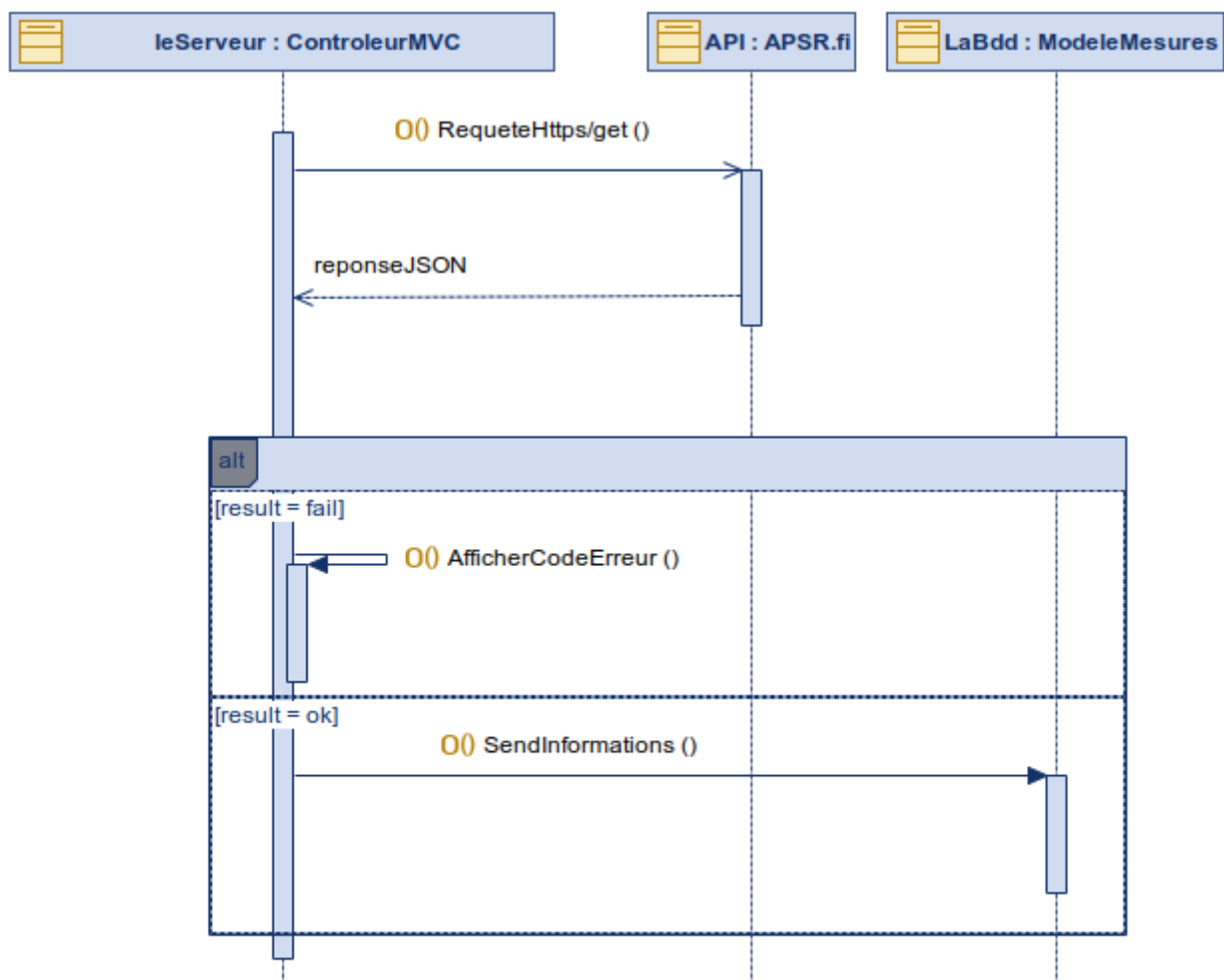
- **Acteur :**
 - Copilote
- **Classes :**
 - Vue
 - API Open Street Map
 - Contrôleur
 - Modèle
- **Pré-condition :**
 - L'application est lancée et la connexion internet est active.
- **Post-condition :**
 - L'interface affiche une carte avec les marqueurs du véhicule et du ballon à jour.



Cas d'Utilisation N°2 et 3: Mettre à jour la base de donnée & Récupérer les positions

Description : Mettre à jour la base de données et récupérer les positions de la voiture et du ballon.

- **Classes :**
 - API APSR.fi
 - Contrôleur
 - Modèle
- **Pré-condition :**
 - Le service externe APRS.fi est disponible & le serveur est en marche.
- **Post-condition :**
 - [APRS.fi](#) result = fail.



3.3. Matrice RACI

Cette matrice établit les rôles et les responsabilités (Responsable, Exécutant, Consulté, Informé) concernant les activités principales du projet.

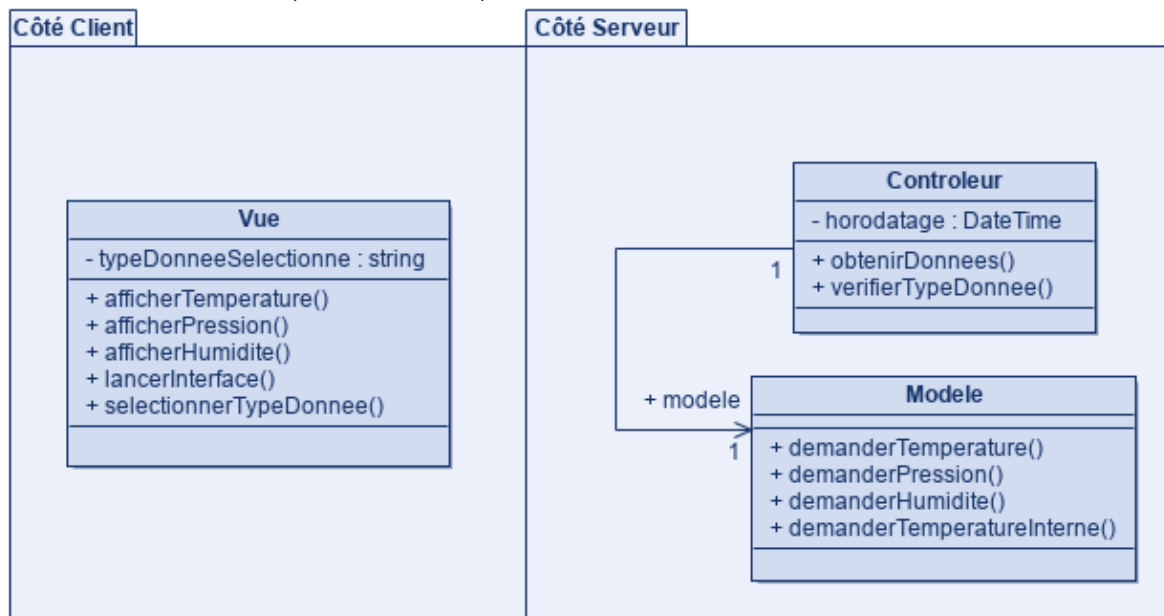
Définition des Rôles RACI

Rôle	Désignation	Description
R	Responsable (Responsible)	Personne ou entité qui exécute la tâche. C'est elle qui réalise le travail nécessaire à l'achèvement de l'activité. Dans la matrice, il peut y avoir plusieurs « R ».
A	Approbateur (Accountable)	Personne ou entité qui est ultimement responsable de l'achèvement correct et complet de la tâche. Il ne peut y avoir qu'un seul « A » par tâche ou activité. Il a l'autorité finale d'approbation.
C	Consulté (Consulted)	Personne ou entité dont l'opinion ou l'expertise est requise pour l'exécution de la tâche. Il y a une communication bidirectionnelle .
I	Informé (Informed)	Personne ou entité qui est tenue informée de l'avancement ou du résultat de la tâche. Il y a une communication unidirectionnelle .

3.4. Diagrammes de Classes

Figure 3.4.1-1

Diagramme de Classes "Visualiser" - Modélisation des classes logicielles permettant de visualiser les données (Interface Web)

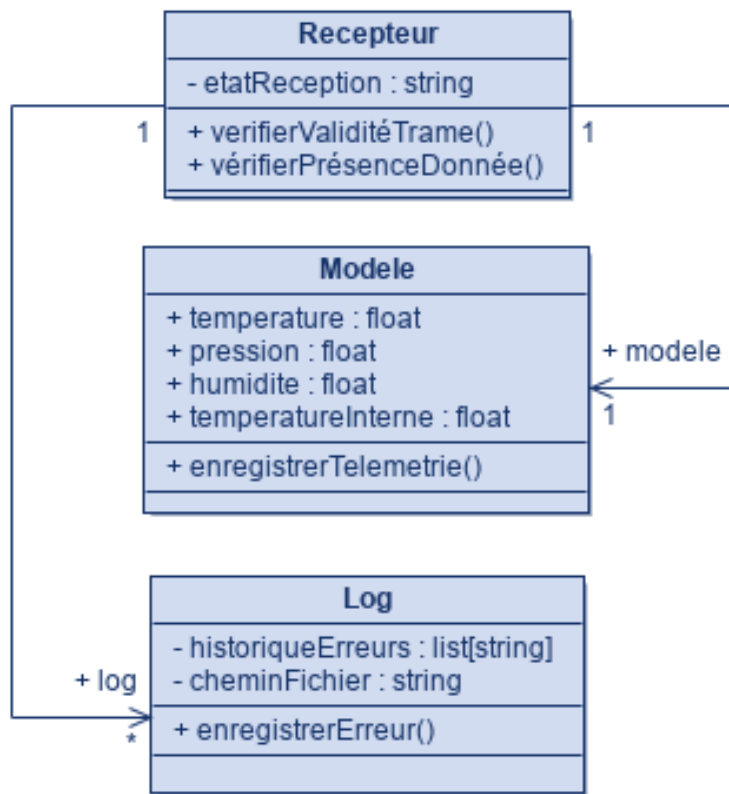


Fonction/Méthode	Explication
Vue	
afficherTemperature()	affiche la température sur l'interface
afficherPression()	affiche la pression sur l'interface
afficherHumidite()	affiche l'humidité sur l'interface
lancerInterface()	démarrage de l'interface
selectionnerTypeDonnee()	sélectionner le type de donnée (click sur un bouton pour choisir le type)
Controleur	
obtenirDonnees()	obtient les données depuis la bdd
verifierTypeDonnees()	vérifie que les données ont bien été reçues
Modele	
demanderTemperature()	demande la température à la bdd
demanderPression()	demande la pression à la bdd
demanderHumidite()	demande l'humidité à la bdd
demanderTemperatureInterne()	demande la température interne directement à la nacelle

Figure 3.4.1-2

Diagramme de Classes "Recevoir"

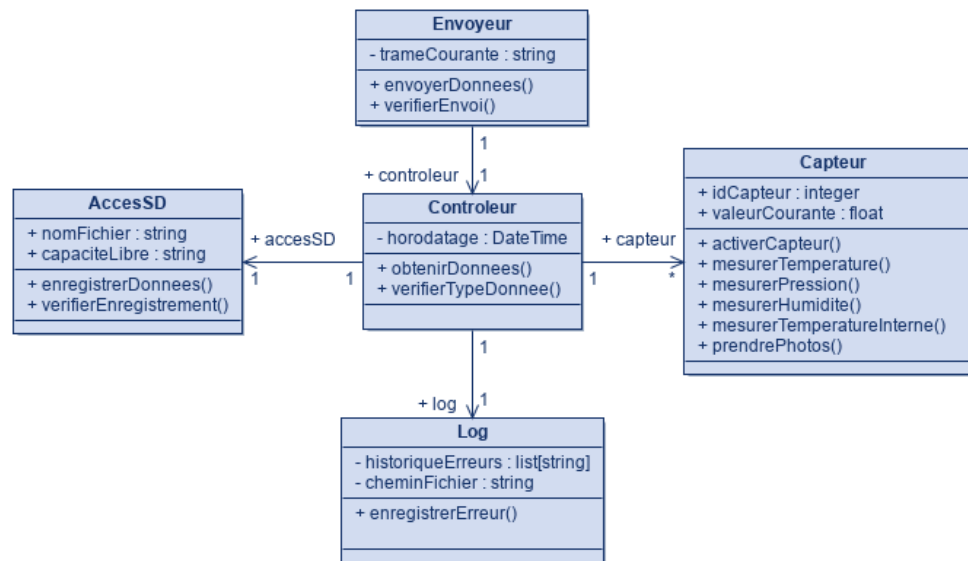
- Modélisation des classes logicielles permettant de recevoir les données



Fonction/Méthode	Explication
Recepteur	
verifierValiditeTrame()	vérifie que la trame reçue n'est pas corrompue
verifierPresenceDonnee()	vérifie individuellement que chaque type de télémétrie est présente
Modele	
enregistrerTelemetrie()	enregistre la télémétrie dans la bdd
Log	
enregisterErreur	enregistre une erreur dans le fichier log

Figure 3.4.1-3

Diagramme de Classes "Ballon" - Modélisation des classes logicielles permettant de mesurer avec les capteurs et d'envoyer les données



Fonction/Méthode	Explication
Envoyeur	
envoyerDonnees()	envoi les données à la station sol
verifierEnvoi()	vérifie que l'envoi s'est bien effectué
AccesSD	
enregistrerDonnees()	enregistre les données sur la carte SD
verifierEnregistrement()	vérifie que cet enregistrement s'est bien passé
Controleur	
obtenirDonnees()	obtient les données du capteur demandé
verifierTypeDonnee()	vérifie le type de donnée obtenue
Capteur	
activerCapteur()	active le capteur demandé
mesurerTemperature()	capteur de température BME280
mesurerPression()	capteur de pression BME280
mesurerHumidite()	capteur d'humidité BME280
mesurerTemperatureInterne()	capteur de température Interne LM75
Log	
enregistrerErreur()	enregistre un message d'erreur dans le fichier log

Figure 3.4.2-1

Diagramme de Classes "Au sol" - Modélisation des classes logicielles pour la gestion de la station sol (réception, décodage APRS, stockage BD, et interface utilisateur).

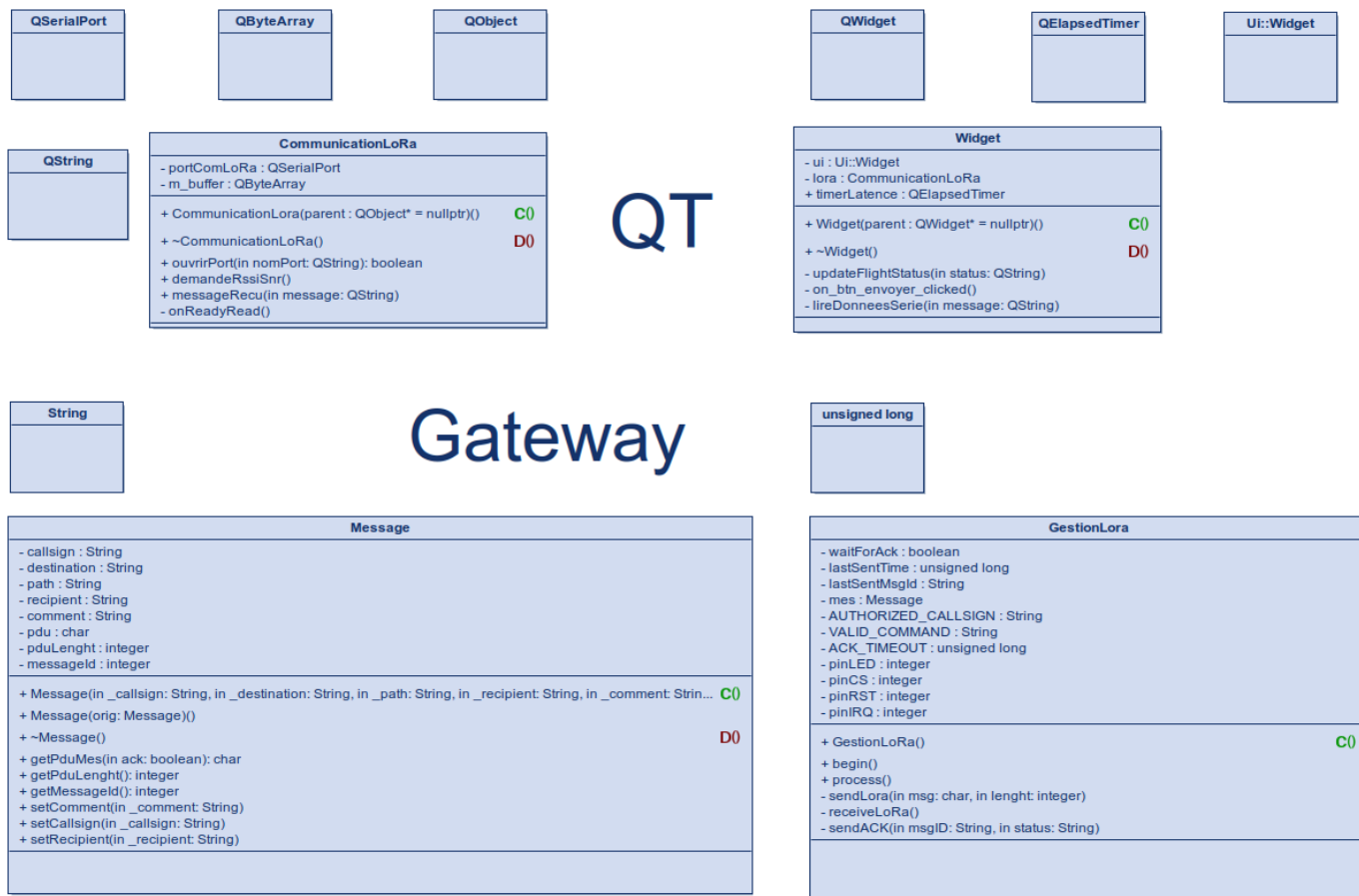


Figure 3.4.2-2

Diagramme de Classes "Embarqué" - Modélisation des classes logicielles gérant l'acquisition de données, la logique de vol, le stockage et la transmission radio (LoRa/SSTV) à bord de la nacelle

MPU6050	
<pre><<Énumérations>> + Sensibility {FS_2G, FS_4G, FS_250DPS, ...} + Dlpf {DLPF_260, DLPF_184, DLPF_5, ...} <<Attributs Privés>> - deviceI2C : i2c* - gpio_int : int - sensibilityAcc : char bil'ttyAcc : - sensibilityGyr : char cal'ttyGyr</pre>	<pre><<Constructeurs & Destructeur>> + MPU6050() + ~MPU6050() + begin(address: int8_t = 0x68) : void + callback_ZMD : void (*)(void)</pre>
<pre><<Attributs Publics (Callbacks d'interruption)>> + callback_FFD : void (*)(void) + callback_ZMD : void (*)(void) + callback_MD : void (*)(void) <<Mesures (Accéléromètre & Gyroscope)>> + getTemperature() : float+ getAccelX() : float + getRotationX() : float + getRotationZ() : float + getAccelX() : float + getAccelZ() : float + getMotion3(ax: int16_t&, ay: int16_t&, az: int16_t&) + getAccelY() : float + getAccelIM() : float + getGyro3(gx: int16_t&, gy: int16_t&, gz: int16_t&) <<Configuration & Calibration>> + setAccSensibility(range : Sensibility) : void + setGyroSensibility: uint8_t : void + setGyroSensibility(range : Sensibility) : void + calibrateA() : void + setDLPFMode(dlpf : Dlpf) : void + calibrateG() : void <<Gestion des Interruptions>> + enableFreeFall(threshold: uint8_t, duration: uint8_t) + disableInterrupt() : void + getStatusInt() : uint8_t <<Méthodes Privées internes>> - meansensorsA(nb: int, mean_ax: int16_t& ...) : void - interruptHandler() : void <<static>></pre>	

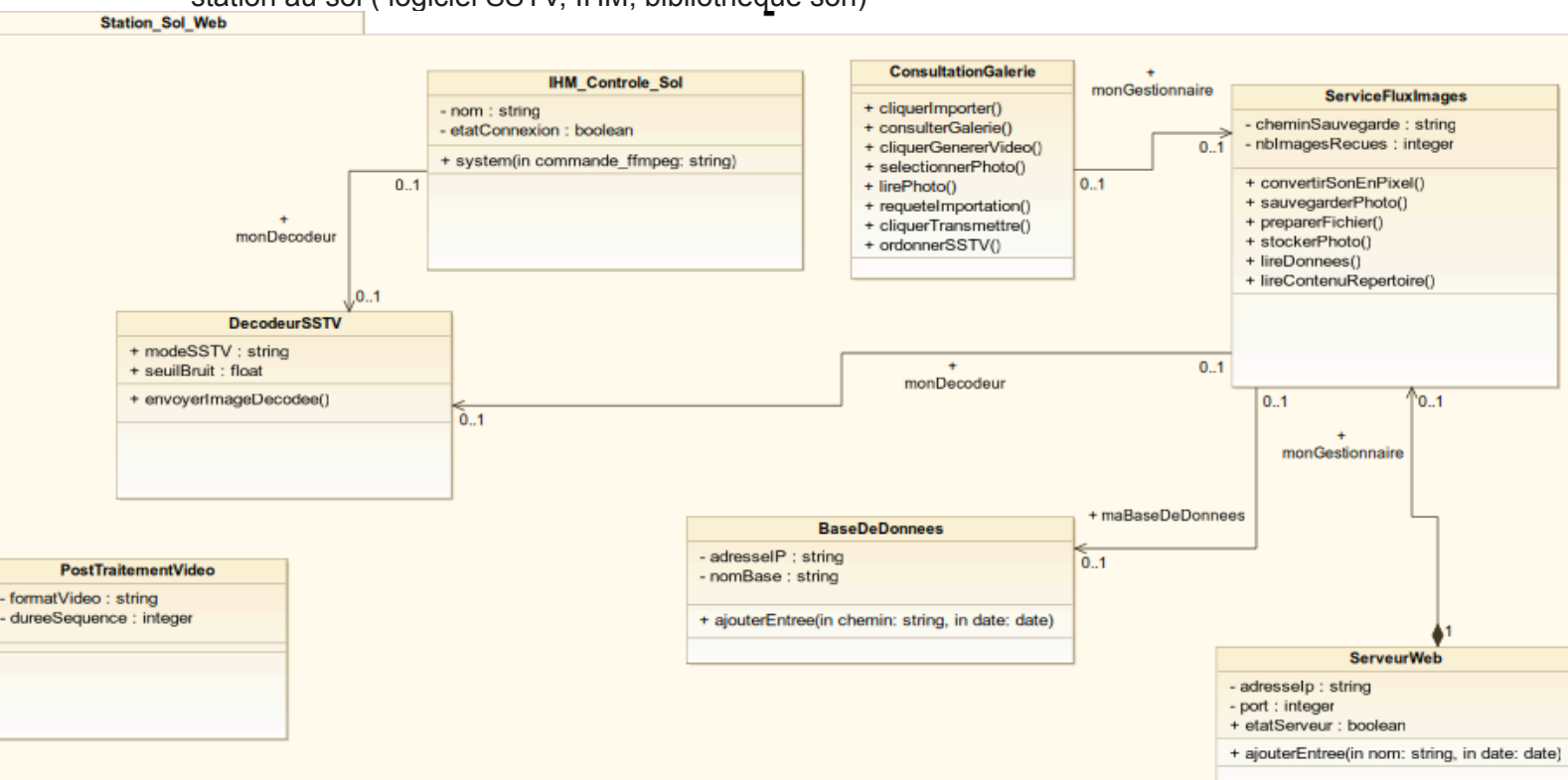
i2c
<pre>- fd : int - error : bool</pre>
<pre><<Constructeurs & Méthodes>> + i2c(adresseI2C : int, idBusI2C : int = 1) + getError() : bool <<Méthodes de Lecture>> + Read() : unsigned char + ReadReg8(reg : int) : unsigned char + ReadReg16(reg : int) : unsigned short + ReadBlockData(reg: int, length: int, values: int*) : int <<Méthodes d'Écriture>> + Write(data : int) : unsigned char + WriteReg8(reg : int, value : int) : unsigned char + WriteReg16(reg : int, value : int) : unsigned short + WriteBlockData(reg: int, length: int, data: int*) : int <<Utilitaires>> + delay_ms(num_ms : unsigned long) : int <<Méthodes Privées internes>> - i2c_smbus_access(rw : char, command : uint8_t, size : int, data : i2c_smbus_data*) : int</pre>

Fonction/Méthode	Explication
AppQT6	
VerifierIntegrité(in trameAPRS: string): boolean	Vérifie l'intégrité de la trame APRS reçue (par exemple, via un checksum) pour s'assurer qu'elle n'est pas corrompue.
VerifierFormat(in trameAPRS: string): boolean	Contrôle si la structure de la trame APRS reçue respecte le format attendu.
LireSNR(): integer	Récupère la valeur du rapport signal sur bruit (Signal-to-Noise Ratio).
LireRSSI(): integer	Récupère la valeur de la force du signal reçu (Received Signal Strength Indicator).

LireTelemetrieData(): string	Extrait les données brutes de télémétrie depuis le système.
DecoderAPRS(in trameAPRS: string): string	Décode et traduit le contenu d'une trame APRS pour en extraire les informations lisibles.
MiseAJourGraphique(in timestamp: long, in mesures: string): string	Prépare et actualise les données mathématiques/logiques en vue de la mise à jour d'un graphique.
EnregistrerReceptionLog(in timestamp: long, in donnees: string)	Transmet les données reçues pour qu'elles soient sauvegardées dans les journaux.
RecevoirTramAPRS(): string	Fonction d'écoute/réception captant une nouvelle trame APRS entrante.
InterfaceUtilisateur	
saisirParametres()	Gère l'action de l'utilisateur lorsqu'il remplit des champs ou paramètres sur l'interface graphique.
validerRequete(in typeReq: string, in valeurs: string): boolean	Vérifie que les données saisies par l'utilisateur pour une requête sont valides avant traitement.
encoderRequeteAPRS(in typeReq: string, in valeurs: string): string	Formate une requête utilisateur classique en une chaîne de caractères compatible avec le protocole APRS.
afficherConfirmation(in message: string)	Fait apparaître un message ou une fenêtre contextuelle validant une action réussie à l'écran.
afficherErreur(in erreur: integer, in description: string)	Affiche un message d'alerte critique sur l'interface, incluant le code d'erreur et les détails.
afficherAvertissement(in typeAvertissement: string, in description: string)	Affiche un message préventif ou non bloquant à destination de l'utilisateur.
AfficherGraphique(in timestamp: long, in mesures: string)	Met à jour visuellement le composant graphique de l'interface avec les nouvelles données de télémétrie.
FichierCSV	
EcrireLogBurst(in timestamp: long, in eventData: string)	Inscrit une ligne dans le fichier CSV correspondant à l'événement d'éclatement du ballon (Burst).
EcrireLogLanding(in timestamp: long, in eventData: string)	Inscrit une ligne dans le fichier CSV signalant l'événement d'atterrissage du ballon (Landing).
EcrireLogRéponse(in timestamp: long, in RSSI: integer, in SNR: integer, in telemetrie: string)	Enregistre une ligne complète de données dans le fichier CSV incluant le temps, la qualité radio et la télémétrie.

Figure 3.4.3-1

Diagramme de Classes "Au sol" - Modélisation des classes logicielles pour gestion de la station au sol (logiciel SSTV, IHM, bibliothèque son)



Fonction/Méthode	Explication
IHM_Control_Sol	
system(string : commande_ffmpeg)	ordonner la commande de création de vidéo
ConsultationGalerie	
cliquerImporter()	importer les photos.
consulterGalerie()	consulter la galerie de photos prise.
cliquerGenererVideo()	créer une vidéo.
selectionnerPhoto()	sélectionner une photo issue de la galerie
ServiceFluxImages	
sauvegarderPhoto()	sauvegarde une photo
DecodeurSSTV	
envoyerImageDecodée()	
Log	
enregistrerErreur()	enregistre un message d'erreur dans le fichier log

Figure 3.4.3-2

Diagramme de Classes "Ballon" - Modélisation des classes logicielles pour l'étude de captures dans le ballon.

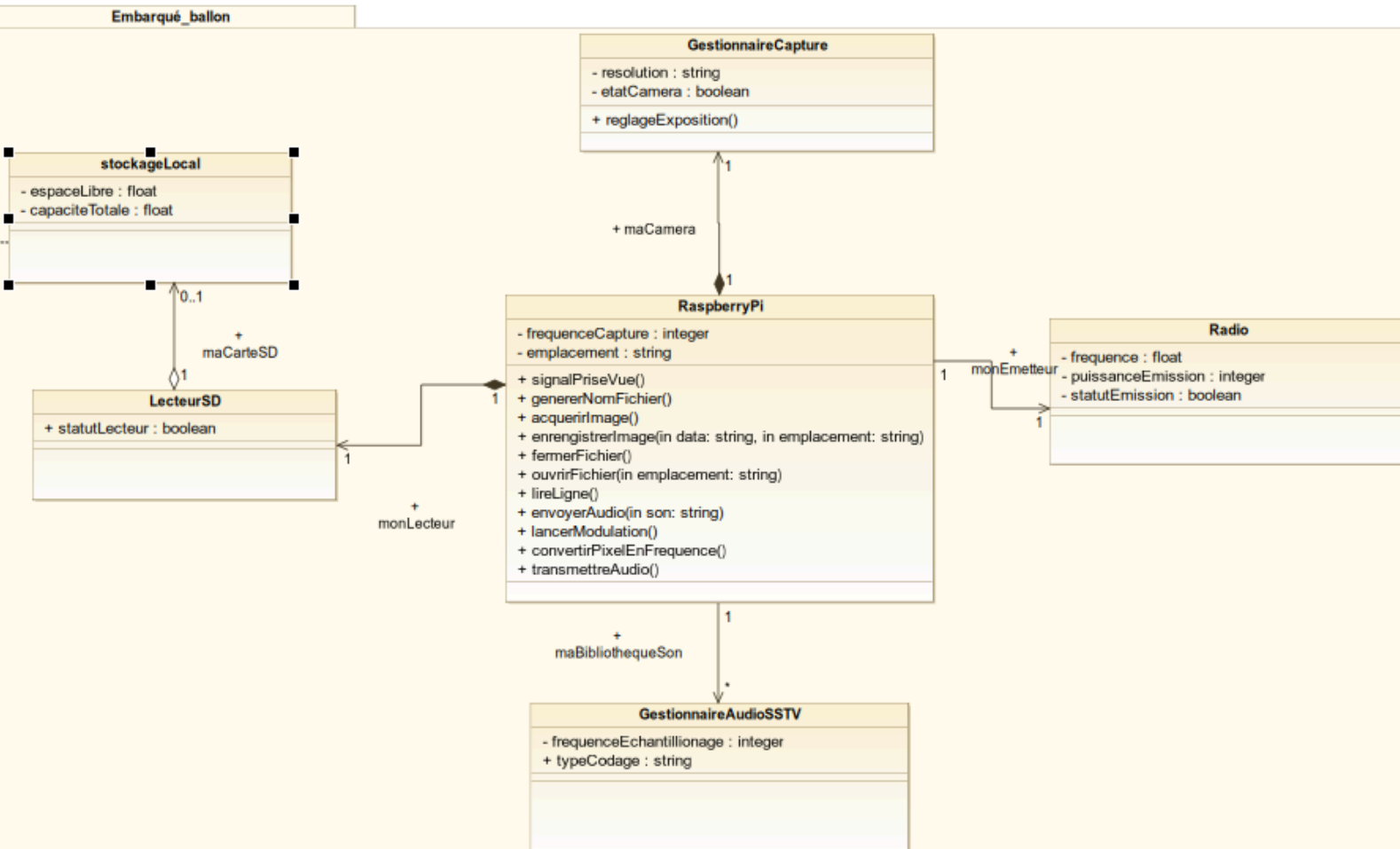


Figure 3.4.4-1

Diagramme de Classes "Site Web" - Modélisation de la structure MVC du site internet permettant la visualisation des positions

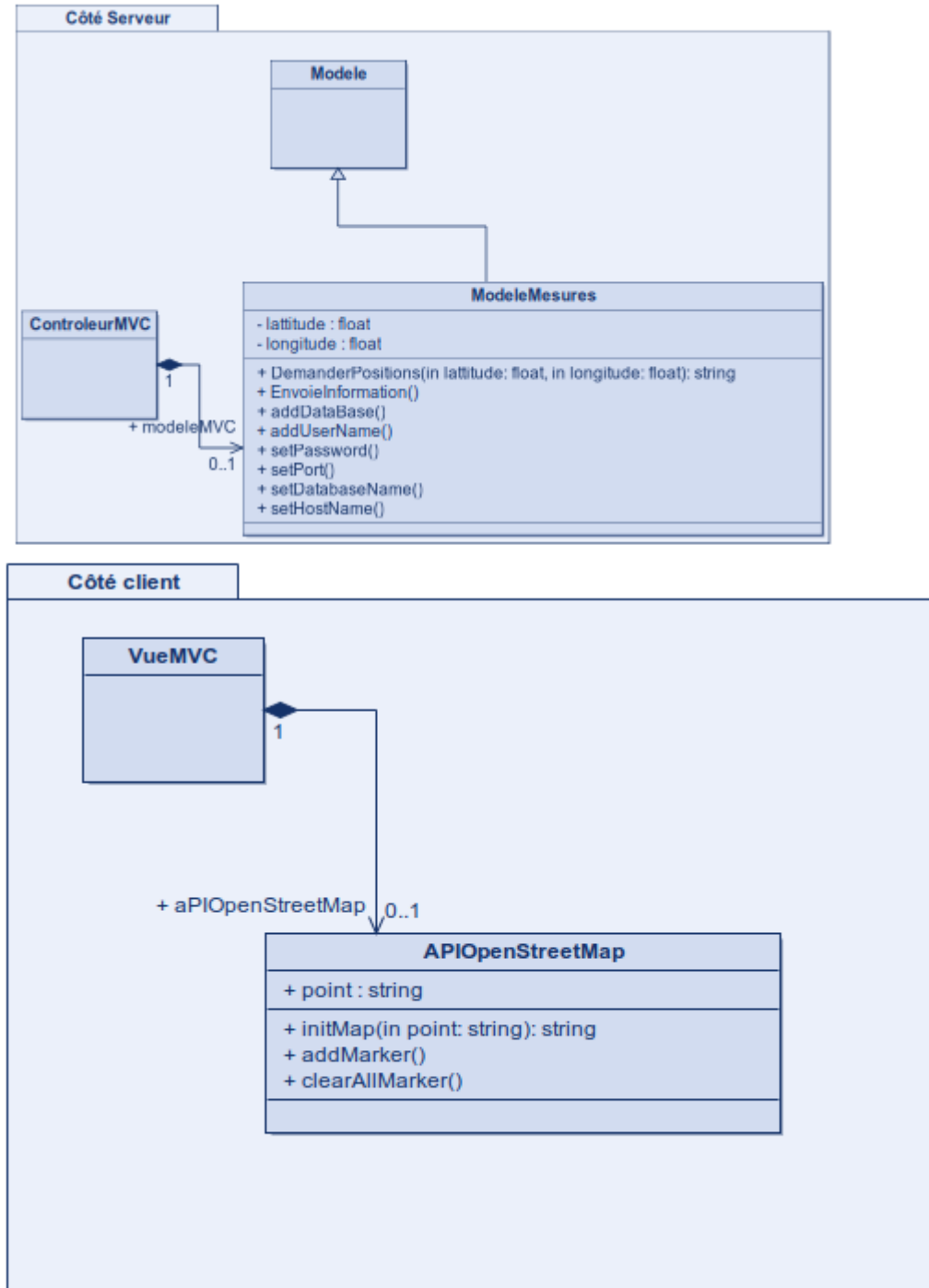
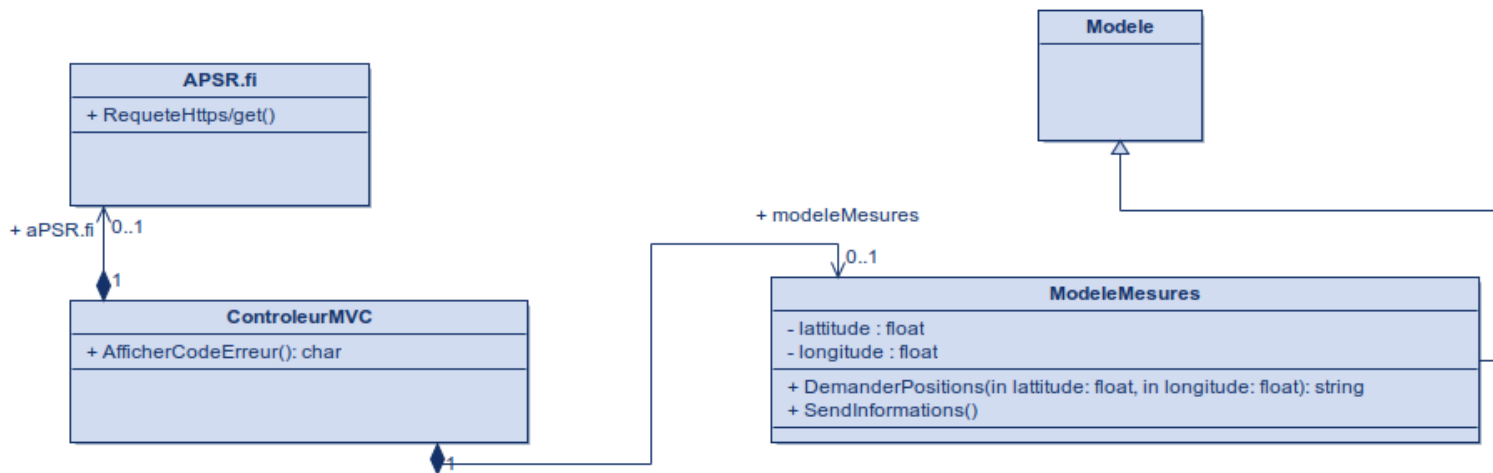
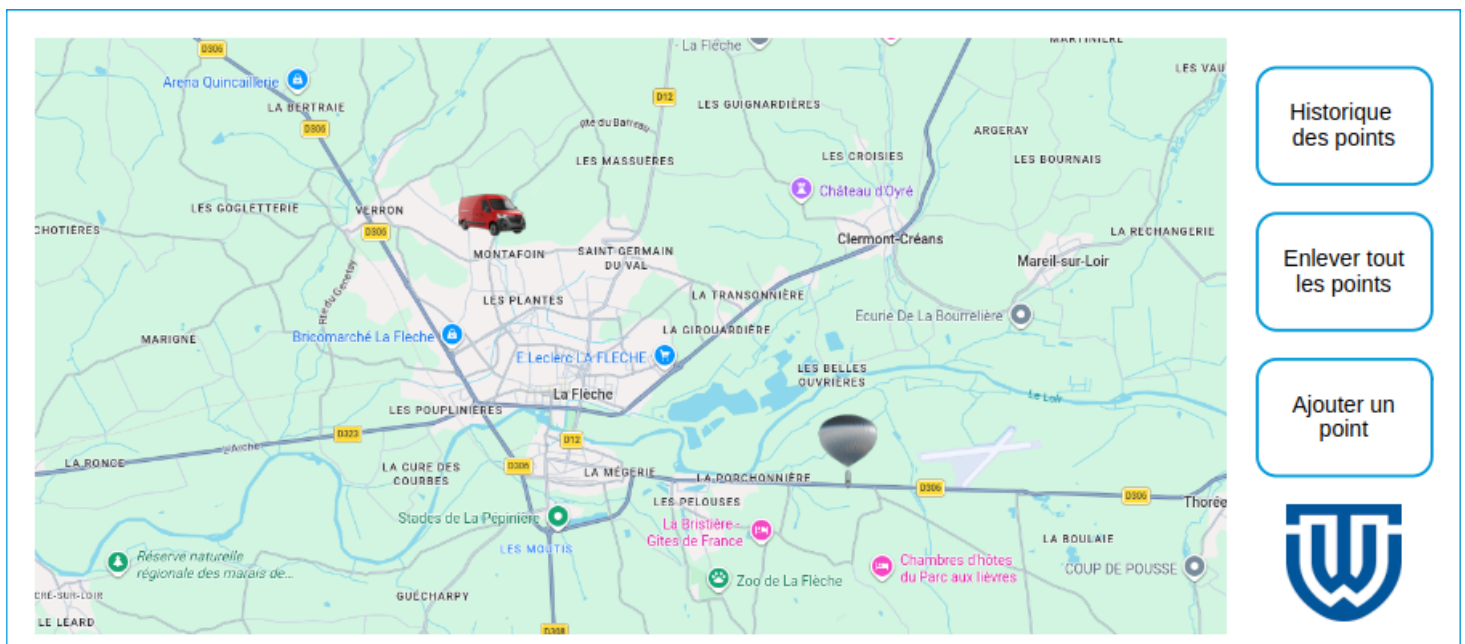


Figure 3.4.4-2

Diagramme de Classes "APRS" - Modélisation de la récupération des positions via [APRS.fi](https://aprs.fi) ainsi que son incrémentation dans la BDD.



Prévision de la vue:



4. Interfaces Utilisateur

4.1. Interface Internaute

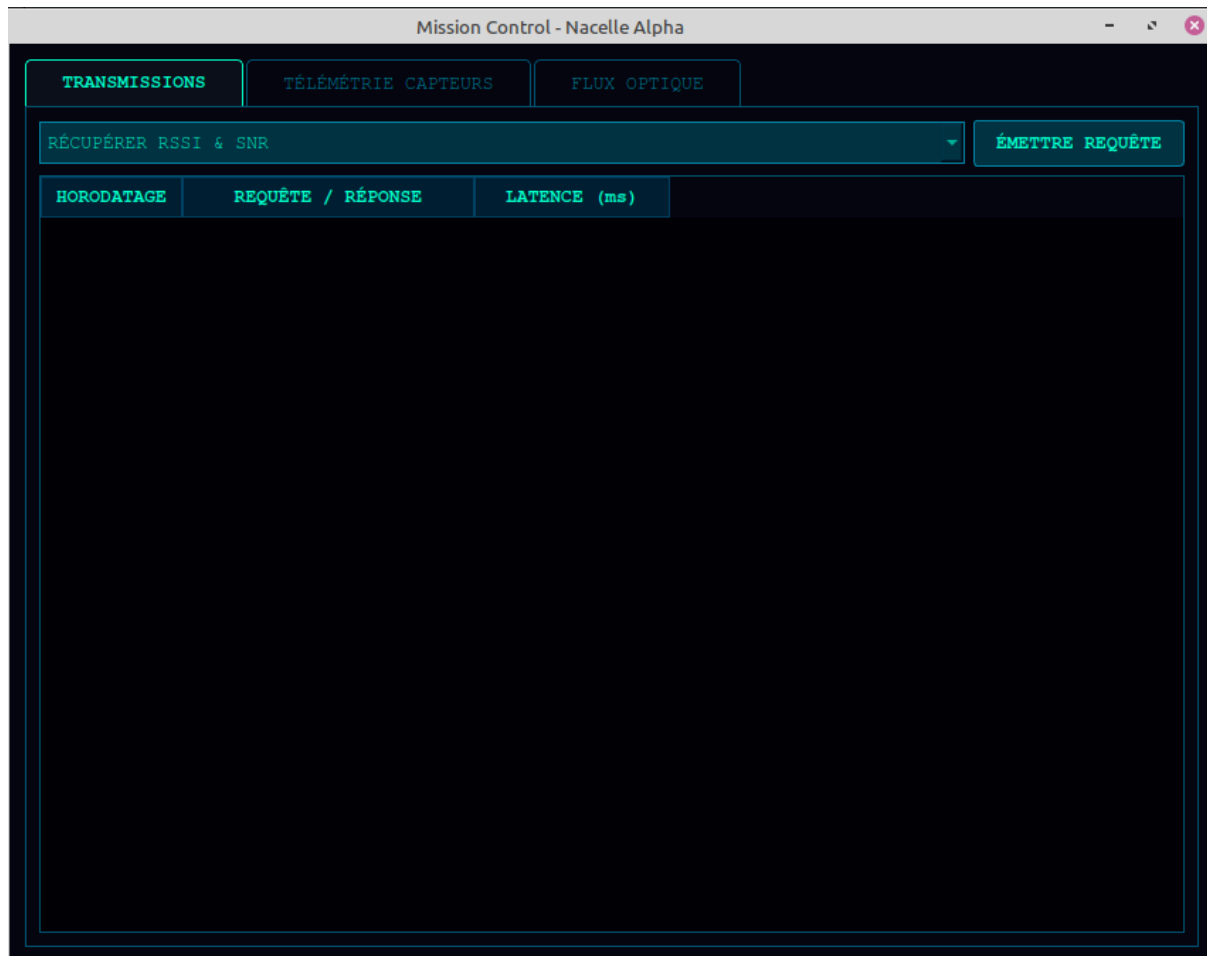
L'interface utilisateur Web est conçue pour être le point d'accès principal aux données et au suivi en temps réel du ballon stratosphérique pour l'internaute et l'opérateur d'analyse. Elle centralise les informations collectées par la nacelle et le système au sol, offrant une vue complète de la mission.

(image interface web , partie photos)

(image interface web , partie télémétrie)

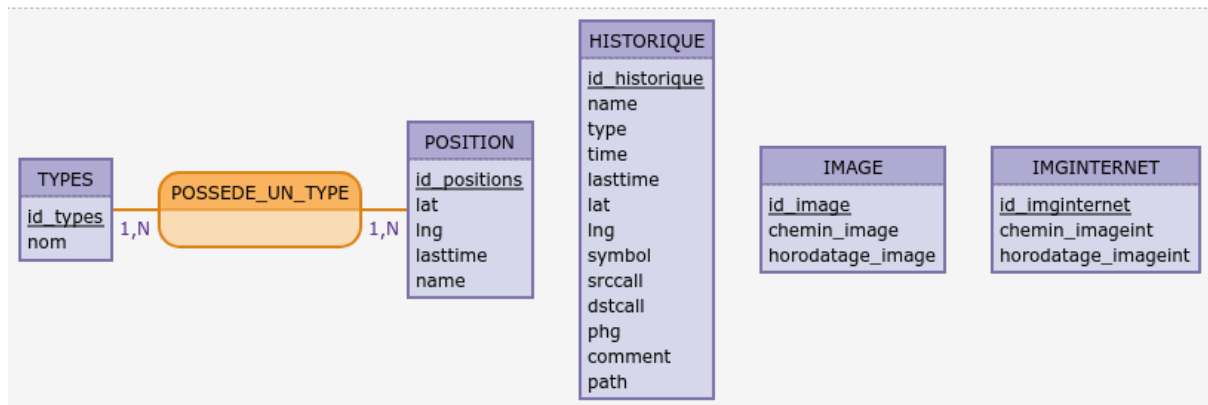
4.2. Interface Opérateur

(texte explicatif)



5. Base de Données

5.1. Diagramme MCD (Modèle Conceptuel de Données)



TYPES: id_types, nom

POSSEDE_UN_TYPE, 1N TYPES, 1N POSITION

POSITION: id_positions, lat, lng, lasttime, name

HISTORIQUE: id_historique, name, type, time, lasttime, lat, lng, symbol, srccall, dstcall, phg, comment, path

IMAGE: id_image, chemin_image, horodatage_image

IMGINTERNET: id_imginternet, chemin_imageint, horodatage_imageint

6. Planification

6.1. Diagramme de Gantt

